

River Publishers Series in Communications

Microcontroller and Smart Home Networks

Dawoud Shenouda Dawoud and Peter Dawoud



River Publishers

Microcontroller and Smart Home Networks

RIVER PUBLISHERS SERIES IN COMMUNICATIONS

Series Editors:

ABBAS JAMALIPOUR

*The University of Sydney
Australia*

MARINA RUGGIERI

*University of Rome Tor Vergata
Italy*

JUNSHAN ZHANG

*Arizona State University
USA*

Indexing: All books published in this series are submitted to the Web of Science Book Citation Index (BkCI), to SCOPUS, to CrossRef and to Google Scholar for evaluation and indexing.

The “River Publishers Series in Communications” is a series of comprehensive academic and professional books which focus on communication and network systems. Topics range from the theory and use of systems involving all terminals, computers, and information processors to wired and wireless networks and network layouts, protocols, architectures, and implementations. Also covered are developments stemming from new market demands in systems, products, and technologies such as personal communications services, multimedia systems, enterprise networks, and optical communications.

The series includes research monographs, edited volumes, handbooks and textbooks, providing professionals, researchers, educators, and advanced students in the field with an invaluable insight into the latest research and developments.

For a list of other books in this series, visit www.riverpublishers.com

Microcontroller and Smart Home Networks

Dawoud Shenouda Dawoud

International University of East Africa
Uganda

Peter Dawoud

Microsoft
USA



Published 2020 by River Publishers
River Publishers
Alsbjergvej 10, 9260 Gistrup, Denmark
www.riverpublishers.com

Distributed exclusively by Routledge
4 Park Square, Milton Park, Abingdon, Oxon OX14 4RN
605 Third Avenue, New York, NY 10017, USA

Microcontroller and Smart: Home Networks / by Dawoud Shenouda Dawoud,
Peter Dawoud.

© 2020 River Publishers. All rights reserved. No part of this publication may be reproduced, stored in a retrieval systems, or transmitted in any form or by any means, mechanical, photocopying, recording or otherwise, without prior written permission of the publishers.

Routledge is an imprint of the Taylor & Francis Group, an informa business

ISBN 978-87-7022-156-6 (print)

While every effort is made to provide dependable information, the publisher, authors, and editors cannot be held responsible for any errors or omissions.

To Nadia, Dalia, Dina, Bahya, and Adam

D.S.D.

To Dajana

P.D



Taylor & Francis

Taylor & Francis Group

<http://taylorandfrancis.com>

Contents

Preface	xxv
Acknowledgment	xxix
List of Figures	xxxi
List of Tables	xli
List of Abbreviations	xliii

I Inter-integrated Circuits (IIC)	1
--	----------

1 Inter-integrated Circuits (IIC/I2C)	3
1.1 Introduction	3
1.1.1 I2C Revisions	4
1.2 I2C Bus Terminology	6
1.3 I2C Protocol	6
1.3.1 Transaction Format–Message Format	6
1.3.2 Timing Diagram	7
1.4 The I2C Bus Hardware Structure	8
1.4.1 Electrical Consideration	9
1.4.1.1 Types of devices that can be connected to I2C bus	9
1.4.1.2 Electrical considerations that limits I2C bus length	10
1.5 The Physical Bus–I2C Bus Interface	13
1.6 SDA and SCL Signals	15
1.7 Masters and Slaves	15
1.7.1 Buffering and Multiplexing	16
1.8 I2C Data Validity	17
1.9 Voltage Levels and Resistor Values	17

1.10	Start/Stop Sequence	18
1.11	Repeated START Condition	19
1.12	Addressing Structure	19
1.12.1	7-bit Addressing	19
1.12.2	Acknowledge Scheme	20
1.12.3	I2C Addresses Standard: 10-bit Addressing	22
1.12.3.1	I2C bus transactions in case of 10-bit address	23
1.12.4	I2C Addresses Standard: Special Addresses and Exceptions in 7-bit Address Space	25
1.12.4.1	Reserved and none-reserved addresses in 7-bit address space	25
1.12.4.1.1	Reserved addresses in 7-bit address space	25
1.12.4.2	Non-reserved addresses in 7-bit address space	30
1.13	I2C Bus Transaction	30
1.13.1	I2C Bus Events 1: Master (Transmitter) to Slave (Receiver) Data Transfer	32
1.13.2	I2C Bus Events 2: Slave (Transmitter) to Master (Receiver) Data Transfer	33
1.13.3	I2C Bus Events 3: Bidirectional Read and Write in Same Data Transfer	35
1.14	Clock Stretching	36
1.15	Possible Modifications on the Timing Diagram	37
1.16	Bus Clear	38
1.17	Applicability of I2C Bus Features	39
1.18	I2C Modes: Bus Speeds	39
1.18.1	Low-speed Mode or Standard Mode	40
1.18.2	Enhanced I2C (Fast Mode)	40
1.18.3	Fast-mode plus (FM+)	41
1.18.4	High-speed Mode (HS mode)	42
1.18.4.1	Electrical characteristics of HS mode	42
1.18.4.2	Transmission format of high-speed mode	44
1.18.5	Ultra-fast Mode (UFm)	46
1.19	I2C as a Multi-master Bus: Bus Arbitration	46
1.19.1	Arbitration	47
1.19.2	Bus Monitoring	48

1.19.3	Possibility of Collision	49
1.19.4	Clock Synchronization and Handshaking	49
1.19.4.1	Clock synchronization	49
1.19.4.2	Handshaking: Using the clock synchronizing mechanism as a handshake	50
1.20	I2C Interface: Connecting I2C Bus to a PC	52

2 Design of I2C Bus and Operation 55

2.1	Design of I2C Bus	55
2.1.1	Open-drain Lines	55
2.1.1.1	Open-drain for Bidirectional Communication	55
2.1.2	Calculation of the Pull-up Resistor	57
2.1.2.1	Supply voltage (V_{CC})	58
2.1.2.2	Total bus capacitance (C_{BUS} or C_b) and $R_p(\max)$	59
2.1.2.3	Total high-level input current (I_{IH}): Input leakage	63
2.1.2.4	Bus speed versus power consumption	64
2.1.3	Maximum Clock Frequency of I2C Bus	65
2.1.3.1	Using repeaters: Effect of increasing the number of devices—use of repeaters	65
2.1.3.2	I2C bus without a repeater	65
2.1.3.3	I2C bus with a repeater	67
2.1.4	Series Protection Resistors	71
2.1.4.1	Effect of the serial resistance on static low level of I2C line	71
2.1.4.2	Serial resistance and debugging	73
2.1.4.3	Termination versus capacitance	74
2.1.5	Cross Channel Capacitance: Crosstalk between SDA and SCL	74
2.2	Operating above the Maximum Allowable Bus Capacitance	76
2.2.1	Reduced f_{SCL}	76
2.2.2	Higher Drive Outputs	78
2.3	Managing the Delays in Large Systems: Use of Buffers, Repeaters, Multiplexers and Switches	80
2.4	Bus Buffers, Repeaters, Multiplexers, and Switches	82
2.4.1	Use of Buffers/Repeaters	82
2.4.2	Use of Multiplexors and Switches	83
2.4.3	Switched Pull-up Circuit	84

2.5	Use of Multiplexor/Switches to Solve Address Conflict . . .	86
2.5.1	Use of Multiplexor to Solve Address Conflict . . .	86
2.5.2	Use of Switches to Solve the Slave Address Conflict	87
2.5.3	Control Register	89
2.5.4	Level Shifters	90
2.5.5	Switches and Level Shifting	90
2.6	Advantages and Limitation of I2C Communication	91
2.7	Common Problems in I2C Bus Systems	94
2.7.1	Obscure Problems in Systems	94
2.7.1.1	Analyzing obscure problems	95
2.7.2	Typical Problems on the I2C Bus	95
2.7.2.1	Blocked I2C Bus	95
2.7.2.2	No acknowledge from I2C slave	96
2.7.2.3	Master reports arbitration lost	97
2.7.2.4	Data bytes from slave are 0xff	97
2.8	Case Study: I2C in AVR	97
2.8.1	AVR Atmega32 I2C Registers	98
2.8.1.1	TWBR (TWI bit rate register)	98
2.8.1.2	TWSR (TWI status register)	99
2.8.1.3	TWCR (TWI control register)	100
2.8.1.4	TWDR (TWI data register)	101
2.8.1.5	TWAR [TWI (slave) address register]	102
2.8.2	Programming of the AVR TWI in Master Operating Mode	102
2.8.2.1	Initialize the TWI in master operating mode	102
2.8.2.2	Transmit START Condition	102
2.8.2.3	Send Data	103
2.8.2.4	Receive Data	103
2.8.2.5	Transmit STOP Condition	103
2.8.3	Use I2C/TWI (Two-wire Interface) in AVR ATmega32	103
2.8.3.1	Circuit diagram	104
2.8.3.2	Code explanation for MASTER controller	104
2.8.3.3	Code explanation for slave controller	109

3	Improved Inter-integrated Circuits (I3C)	113
3.1	Derivative Technologies	113
3.2	Improved Inter-integrated Circuits (I3C)	114
3.2.1	Challenges with Existing I2C-based Architectures	115
3.2.2	MIPI I3C Interface	117
3.2.3	I3C Bus Configuration and Device Roles	119
3.2.4	I3C Features	120
3.2.4.1	Dynamic addressing	120
3.2.4.2	Provisional ID	122
3.2.4.3	Possibility of collision during dynamic address assignment (DAA)	123
3.2.4.4	Rule of CCC in dynamic address assignment	123
3.2.4.5	Hot-join	124
3.2.4.6	In-band Interrupt (IBI)	124
3.2.4.6.1	Interrupts	125
3.2.5	Multiple Masters	126
3.2.6	Increased Throughput	127
3.2.7	Power Consumption	128
3.2.7.1	Pull-up resistor	128
3.2.7.2	High-keeper	129
3.2.8	I3C Bus Activity States	129
3.2.9	I3C Bus Conditions When the Bus Is Considered Inactive	129
3.2.10	Time-stamping Capability Defined in the I3C Bus	130
3.2.11	Error Detection and Recovery Methods in I3C	130
3.2.12	Maximum Capacitance Load Allowed on the I3C Bus	131
3.2.13	Maximum Wire Length for I3C Communication	131
3.2.14	Repeaters and I3C	131
3.2.15	Resolving Communication Conflicts on the I3C Bus	131
3.2.16	Can I3C Devices Cause the Communication Bus to “Hang”?	132
3.2.17	MIPI I3C Specifications and Software Development	132

3.2.18 Expected New Features for MIPI I3C	133
Summary	133
References	134

II Controller Area Network (CAN) 135

4 Controller Area Network (CAN) 137

4.1 Introduction to CAN: Overview	137
4.1.1 What is CAN?	137
4.1.1.1 Meaning of Message-based Protocol	138
4.1.2 The Benefits of CAN Bus in Automobile Industry	139
4.1.3 CAN History	140
4.1.3.1 Higher layer implementations: Higher layer protocols (HLPs)	142
4.1.4 Error Detection and Security in CAN	143
4.1.5 The Application of CAN Bus	143
4.2 CAN Standard	144
4.2.1 CAN Architecture: OSI 7 layer reference model and CAN model	144
4.2.2 Higher Layer Protocols (HLPs)	148
4.3 CAN Features and Characteristics	148
4.3.1 Some Features of CAN Bus	148
4.3.2 Key characteristics of CAN	149
4.4 CAN Physical Layer	151
4.4.1 CAN Bus Description	151
4.4.1.1 Electrical consideration	153
4.4.1.1.1 Inverted logic of CAN bus	154
4.4.1.1.2 Robustness	155
4.4.1.1.3 Bus lengths	155
4.4.1.1.4 Bit rates and bus lengths	156
4.4.1.1.5 Propagation delay	157
4.4.1.1.6 Cables	158
4.4.1.1.7 Shield termination	159
4.4.1.1.8 Grounding	159
4.4.1.1.9 Bus termination—line termination	159
4.4.1.1.10 Connectors	160

4.4.2	CAN Node	160
4.4.2.1	CAN nodes types	162
4.4.3	Sublayers of the CAN Physical Layer	164
4.4.3.1	The physical coding sublayer: Overview	164
4.4.3.1.1	Bit-timing setting for Standard CAN	165
4.4.3.1.1.1	The Can Bit Time	165
4.4.3.1.1.2	Sample point	167
4.4.3.1.1.3	Implementation of bit segments in practical CAN controllers	168
4.4.3.1.1.4	Information processing time (IPT)	168
4.4.3.1.2	Bit timing setting for CAN FD	168
4.4.3.2	Calculation of baud-rate and sample point	169
4.4.3.3	Synchronization jump width (SJW)	171
4.4.3.4	Bit timing control registers	171
4.4.3.5	Synchronizing the bit time	171
4.4.3.6	Oscillator tolerance	173
4.4.3.7	Propagation delay	173
4.4.3.8	Synchronization: Synchronization Mechanisms used in CAN	175
4.4.3.9	Calculating oscillator tolerance for SJW	179
4.4.3.10	Configuring the bit	180
4.4.3.11	The physical media attachment sublayer	182
4.4.3.12	The medium-dependent sublayer	183
4.4.4	Maximum Number of Nodes	185
4.5	CAN Transceiver	185
4.5.1	CAN Transceiver Features	185
4.5.1.1	Supply voltage	185
4.5.1.2	High short-circuit protection	186

4.5.1.3	High ESD protection	186
4.5.1.4	High input impedance	187
4.5.1.5	Controlled driver output transition times	187
4.5.1.6	Low current standby and sleep modes	187
4.5.1.7	Thermal shutdown protection	188
4.5.1.8	Glitch free power up and power down . .	188
4.5.1.9	Unpowered node does not disturb the bus	188
4.5.1.10	Reference voltage	188
4.5.1.11	V-Split	189
4.5.1.12	Loopback	189
4.5.1.13	Autobaud loopback	190
4.5.2	CAN Transceiver Example: MCP2551 CAN TRANSCIVER	190
4.5.2.1	General MCP2551 Operation	190
4.5.2.1.1	Transmit	190
4.5.2.1.2	Receive	191
4.5.2.1.3	Recessive STATE	191
4.5.2.1.4	Dominant STATE	191
4.5.2.2	Modes of operation	191
4.5.2.2.1	HIGH SPEED	191
4.5.2.2.2	SLOPE CONTROL	192
4.5.2.2.3	STANDBY	192
4.5.2.3	Permanent dominant detection on transmitter	192
4.5.2.3.1	Power-on reset and brown-out	193
4.5.2.3.2	Power-On Reset (POR)	193
4.5.2.3.3	Brown-out detection (BOD)	194
4.5.2.3.4	Ground offsets	194

5	CAN Data Link Layer	197
5.1	Data Link Layer	197
5.1.1	CAN Communication Services	198
5.1.1.1	Communication services	198
5.1.1.2	Remote transmission requests (RTR): Read object service	200

5.1.2	Multiple Bus Access	200
5.1.2.1	Meaning of CSMA/CD+AMP	201
5.1.2.2	Bus arbitration	203
5.1.3	Standard CAN or Extended CAN	205
5.1.3.1	The bit fields of standard CAN and extended CAN	206
5.1.3.2	Standard (base) CAN frame	206
5.1.3.3	Extended frame format	212
5.1.4	CAN Messages	213
5.1.4.1	CAN Frames	213
5.1.4.1.1	Data frame	214
5.1.4.1.2	Remote data frame	215
5.1.4.1.3	Error frame	217
5.1.4.1.4	Overload frame	219
5.1.4.1.5	Bit stuffing	221
5.2	CAN Bus Error Handling	222
5.2.1	Error Detection Methods	223
5.2.2	Form Error	224
5.2.3	CRC Error	225
5.2.4	Acknowledge Error	226
5.2.5	Stuff Error	226
5.2.6	Bit Error	227
5.2.7	Local Errors in EOF	227
5.2.8	Message Doubling	228
5.2.9	Error States of CAN Node	229
5.2.10	Transmit Error Counter	231
5.2.11	Receive Error Counter	232
5.2.12	Recovery from Bus Off	232
5.2.13	Error Confinement Mechanisms	233
5.2.14	Bus Loading	237
5.2.15	Time-triggered Protocols	238
5.3	CAN FD—The Basic Idea	239
5.4	ISO 11898 Review and Optional Modes	243
5.4.1	Bus Monitoring Mode	243
5.4.2	Time-Triggered Communication (TTC)	243
5.5	CAN in Action	244
	References	247
	External Links	249

III	ZigBee	251
6	IEEE 802.15.4 and ZigBee	253
6.1	Introduction	253
6.1.1	ZigBee General Description	253
6.1.2	ZigBee: Overview	253
6.1.3	Evolution of LR-WPAN Standardization	255
6.1.4	ZigBee History	256
6.1.5	ZigBee Application Profiles	257
6.2	ZigBee Wireless Sensor and Control Network	259
6.2.1	ZigBee Network Characteristics	259
6.2.1.1	ZigBee versus Bluetooth	260
6.2.2	ZigBee Device Types and Operating Modes	261
6.2.2.1	Operating modes	265
6.2.3	ZigBee Topologies	266
6.2.3.1	Forming the ZigBee network	267
6.2.3.2	Joining the ZigBee Network	267
6.2.4	End Device Addressing	268
6.2.5	Depth of a Network, Number of Children, and Network Address Allocation	268
6.3	ZigBee Protocol Stack Overview	269
6.3.1	Application Layer (APL)	271
6.3.1.1	ZigBee Device Objects (ZDO)	271
6.3.1.2	APS (application support sublayer)	272
6.3.1.2.1	ZigBee APS frame format	273
6.3.1.2.2	APS Ack frames	273
6.3.1.2.3	APS command frames	273
6.3.1.3	Application framework	273
6.3.2	Network Layer (NWK)	275
6.3.2.1	ZigBee network layer frame format	276
6.3.3	Security Services	276
6.3.3.1	Basic security model	277
6.3.3.2	Security architecture	277
6.3.3.3	Network layer security	278
6.3.3.4	APL layer security	278
6.3.3.5	Trust center role	279
6.3.4	ZigBee Address Assignment	279

6.4	IEEE 802.15.4 Standard	281
6.4.1	IEEE 802.15.4 Standard	282
6.4.2	Device Types	283
6.4.3	Network Topology	283
6.4.3.1	Star topology	284
6.4.3.2	Mesh topology, also, peer-to-peer topology	285
6.4.3.3	Tree topology	286
6.4.3.4	Cluster tree topology	287
6.4.4	802.14.5 Architecture	288
6.4.4.1	PHY layer	289
6.4.4.1.1	Function of physical layer in ZigBee architecture	292
6.4.4.1.2	Frequency band, data rate, and channel numbering	293
6.4.4.1.3	IEEE 802.15.4 frequencies and frequency bands	293
6.4.4.1.4	ZigBee frequency bands and data rates	294
6.4.4.1.5	IEEE 802.15.4 modulation formats	296
6.4.4.1.6	ZigBee physical layer frame format-PPDU	297
6.4.4.2	Receiver energy detection (ED)	299
6.4.4.3	Link quality indication (LQI)	299
6.4.4.4	Clear channel assessment (CCA)	300

7 ZigBee Stack Layers 301

7.1	ZigBee Mac Layer	301
7.1.1	Superframe Structure	302
7.1.1.1	Updating superframe configuration	304
7.1.2	CSMA-CA Algorithm	305
7.1.3	Data Transfer Model	307
7.1.4	Network Formation: Starting a PAN	309
7.1.5	Network Joining	313
7.1.5.1	Network discovery	313
7.1.5.2	Network join	314
7.1.5.2.1	Network join-child	314
7.1.5.2.2	Network join-parent	315

7.1.6	Association and Disassociation	315
7.1.6.1	Orphan notification	319
7.1.7	Synchronization	320
7.1.8	Transmission, Reception, and Acknowledgment . .	322
7.1.9	GTS Allocation and Management	323
7.1.10	MAC Layer Frame	325
7.1.10.1	Data frame	329
7.1.10.2	Beacon frame	330
7.1.10.2.1	Beacon generation	331
7.1.10.3	Acknowledgment frame	332
7.1.10.4	MAC command frame	332
7.1.11	Channel Access Mechanism	333
7.1.12	GTS Allocation and Management	334
7.1.13	Related MAC Standards	335
7.2	ZigBee Routing (Network) Layer	336
7.2.1	Broadcasting	337
7.2.2	ZigBee Routing Protocols	340
7.2.2.1	AODV: Ad hoc on-demand distance vector	340
7.2.2.2	AODVjr	345
7.2.2.3	Multicast route	345
7.2.2.4	Dynamic source routing (DSR)	345
7.2.2.5	Tree hierarchical routing protocol (HERA) or ZBR (ZigBee Routing)	348
7.2.2.6	ZigBee cluster label (ZiCL)	353
7.2.2.7	ZigBee multipath hierarchical tree routing (Z-MHTR)	353
7.2.2.8	Neighbor table-based routing techniques	354
7.2.2.8.1	Neighbor table	355
7.2.2.9	Many-to-one routing	356
7.2.2.10	ZigBee RF4CE	356
7.2.2.11	RF4CE multi-star topology	357
7.2.2.12	ZigBee RF4CE pairing process	357
7.2.2.13	ZigBee smart energy V2	357
8	ZigBee PRO and ZigBee SECURITY	359
8.1	ZigBee and ZigBee PRO Feature Sets	359
8.1.1	Management and Addressing	360

	8.1.1.1	Link management	360
	8.1.1.2	Group addressing	360
	8.1.1.3	Compatibility	360
	8.1.1.4	Power management	360
8.1.2		Improved Security	361
	8.1.2.1	Trust center	363
8.1.3		Scalability	363
	8.1.3.1	Stochastic addressing	363
	8.1.3.2	Route aggregation	364
8.1.4		Improved Ease-of-Use and Operation	365
	8.1.4.1	Fragmentation	366
	8.1.4.2	Commissioning	366
8.1.5		Improved Resiliency	366
	8.1.5.1	Asymmetric link	366
	8.1.5.2	Frequency agility	367
	8.1.5.3	PAN ID conflict resolution	368
8.2		ZigBee Security	368
8.2.1		802.15.4 Security: Overview	370
	8.2.1.1	802.15.4 Access control list	374
8.2.2		ZigBee Security	375
	8.2.2.1	ZigBee security services	375
8.2.3		ZigBee Security Models	376
	8.2.3.1	ZigBee security architecture	378
	8.2.3.2	Network layer security	379
	8.2.3.3	APL layer security	380
	8.2.3.4	MAC Layer Security	381
	8.2.3.5	Trust center role	382
8.2.4		ZigBee Security Keys	382
8.2.5		Key Management	386
	8.2.5.1	Security key modification	388
8.2.6		ZigBee Stack Security Measures	389
8.2.7		ZigBee Vulnerabilities	393
	8.2.7.1	Implementation vulnerabilities	394
	8.2.7.2	Protocol Vulnerabilities	395
	8.2.7.3	Security risks	399
	8.2.7.4	Security key weaknesses	399
8.2.8		Measures for Maximizing ZigBee Security	400
	8.2.8.1	Protecting the network key	400
	8.2.8.2	Rolling the network key	402

8.2.8.3	Preventing spoof leave notifications key	403
8.2.8.4	Preventing rejoin security attacks	403
8.2.8.5	Persisting essential data when using cloud services	404
8.2.8.6	Key establishment cluster and security certificates	404
8.2.9	ZigBee Security–Conclusion	405
8.3	Applications of ZigBee Technology	406
8.4	Advantages of ZigBee Technology	407
8.4.1	ZigBee Technology Advantages	407
8.4.2	Disadvantages	408
8.4.3	Conclusion	408
8.5	Future Scope Of ZigBee	409
	References	411

IV Bluetooth **413**

9 Bluetooth **415**

9.1	Introduction: Overview	415
9.1.1	History of Bluetooth	416
9.1.2	Piconets and Scatternets	417
9.1.3	Bluetooth Spectrum	419
9.1.4	Bluetooth Frequency and Connectivity Ranges . .	419
9.1.5	Data Rate and Modulation	419
9.2	Bluetooth Stack Architecture	420
9.2.1	Relation between Classic Bluetooth, OSI model, and 802.11 Wi-Fi	422
9.2.2	Classical Bluetooth Stack	424
9.2.2.1	Bluetooth stack as core and profile . . .	425
9.3	Bluetooth Stack Layers	427
9.3.1	Bluetooth Radio	427
9.3.2	Baseband	428
9.3.3	Link Manager Protocol (LMP)	430
9.3.4	Host Controller Interface (HCI)	431
9.3.5	Logical Link Control and Adaptation Protocol (L2CAP)	431
9.3.6	Radio Frequency Communications (RFCOMM) . .	432

9.3.7	Adapted Protocols	432
9.3.8	Service Discovery Protocol	432
9.3.9	Server-Client Model	433
9.4	Device Discovery	435
9.4.1	Inquiry	435
9.4.2	Page	438
9.4.3	Optional Paging Schemes	441
9.5	Service Discovery and Access	441
9.5.1	Bluetooth Service Discovery Protocol	442
9.5.1.1	Format	442
9.5.1.2	Service record	443
9.5.1.3	Representation of data	444
9.5.1.4	Searching and browsing services	445
9.5.2	Examples of SDP: Universal Plug and Play	446
9.5.2.1	Architecture overview	447
9.5.2.2	Operations overview	447
9.6	Bluetooth Security	448
9.6.1	Basic Means of Providing Bluetooth Security	449
9.6.2	Bluetooth Security and Trust Modes	451
9.6.2.1	Bluetooth security modes	451
9.6.2.1.1	Security mode 2 role	452
9.6.2.2	Bluetooth trust modes	455
9.6.3	Discoverability in Devices	455
9.6.4	Bluetooth Security Services	455
9.6.5	Built-in Security Features	457
9.7	Bluetooth Vulnerabilities and Threats	457
9.7.1	Vulnerabilities in Bluetooth Versions	457
9.8	Reasons for Bluetooth Vulnerabilities	458
9.8.1	Vulnerability to Eavesdropping	459
9.8.2	Potential Weaknesses	459
9.8.2.1	Encryption mechanisms	459
9.8.2.2	Association models of SSP	459
9.8.2.3	Device configuration	460
9.8.2.4	Bad coding during development of RFCOMM stack implementation	460
9.8.2.5	Re-use of older services for different protocols	460
9.8.2.6	IrMC permissions	460
9.9	Common Bluetooth Attacks	460

9.10	Bluetooth Risk Mitigation and Countermeasures	470
9.10.1	Mitigation Techniques	470
9.10.2	Applications for Protecting Bluetooth Devices . . .	472
	References	472
	Sources	478
10	Z-Wave Technology	479
10.1	Introduction: What is Z-Wave?	479
10.1.1	Z-Wave Operating Characteristics	479
10.1.2	Technical Characteristics: Z-Wave frequency bands	480
10.1.3	Z-Wave Features	481
10.1.4	Z-Wave History	481
10.2	Z-Wave Protocol	483
10.2.1	Overview	483
10.2.2	Z-Wave Basic Device Classes: Controller and Slave Nodes	484
10.2.2.1	Controllers	485
10.2.2.1.1	Portable Controller	486
10.2.2.1.2	Static controller	486
10.2.2.1.3	Static update controller . . .	486
10.2.2.1.4	SUC ID server	486
10.2.2.1.5	Installer controller	487
10.2.2.1.6	Bridge controller	487
10.2.2.2	Slaves	487
10.2.2.2.1	Slave	487
10.2.2.2.2	Routing slave	487
10.2.2.2.3	Enhanced slave	488
10.2.2.3	Home ID and Node ID	489
10.2.2.4	Z-Wave chip	489
10.2.2.5	Z-Wave frame structure	490
10.2.2.5.1	Z-Wave messaging	491
10.2.3	Z-Wave Physical Layer	491
10.2.4	Z-Wave MAC Layer	493
10.2.4.1	Z-Wave MAC frame types	493
10.2.4.2	Collision avoidance	494
10.2.5	Z-Wave Transport Layer	496
10.2.5.1	Singlecast frame type	498
10.2.5.2	ACK frame type	499

	10.2.5.2.1	Multicast frame type	499
	10.2.5.2.2	Broadcast frame type	499
10.2.6	Z-Wave Network Layer (Routing Layer)		499
	10.2.6.1	Frame Layout	501
	10.2.6.1.1	Routed singlecast frame type	501
	10.2.6.1.2	Routed acknowledge frame type	501
	10.2.6.2	Routing table	501
	10.2.6.3	Route to node	501
10.2.7	Application Layer		503
	10.2.7.1	Frame layout	504
	10.2.7.2	Node information	506
	10.2.7.2.1	Node information frame flow	506
10.3	Z-Wave Network Operation		506
10.3.1	Z-Wave Network Setup		506
	10.3.1.1	Including and removing device in the network: ‘pairing’ operation	507
	10.3.1.2	Z-wave network enrollment and button associations	507
	10.3.1.3	Button pairs versus on/off toggles	508
10.4	Z-Wave Security		508
10.4.1	Vulnerabilities		510
	10.4.1.1	Impersonation	510
Summary			511
References			512

11 Smart Home Protocols: Comparison 513

11.1	Introduction	513
11.2	ZigBee versus Z-Wave	514
11.2.1	ZigBee versus Z-Wave: Specifications and Capabilities	514
11.2.2	ZigBee versus Z-Wave: The Differences	516
11.2.3	Z-Wave versus ZigBee: The Devices	519
11.3	Z-Wave and ZigBee versus INSTEON	520
11.3.1	INSTEON Overview (www.insteon.com)	520
11.3.2	INSTEON Main Characteristics compared with other Technologies	520

11.3.3	INSTEON versus Z-Wave and ZigBee: Comparison	523
11.3.4	Z-Wave, ZigBee, and INSTEON Compared: Summary	524
11.4	Z-Wave versus THREAD Protocol: Comparison	524
11.4.1	Internet of Things (IoT)	524
11.4.2	Thread Protocol	529
11.4.3	Thread General Characteristics	531
	11.4.3.1 Network stack: Thread versus z-wave stack	532
11.5	Wi-Fi versus INSTEON and Z-Wave	537
11.5.1	Wi-Fi Overview	537
11.5.2	Z-Wave Compared with Wi-Fi	539
11.5.3	INSTEON Compared with Wi-Fi	539
11.6	Bluetooth Compared with Z-Wave, ZigBee, Wi-Fi, and INSTEON	540
11.7	X10 Technology	544
11.7.1	X10: Overview	544
11.7.2	INSTEON Compared with X10	546
11.8	Universal Powerline Bus (UPB)	546
11.8.1	UPB an Overview	547
11.8.2	INSTEON Compared with UPB	548
11.9	Intellon	548
11.9.1	Intellon: Overview	548
11.9.2	Intellon Compared with INSTEON Technology . .	549
	References	549

Index	553
--------------	------------

About the Authors	559
--------------------------	------------

Preface

During the last few decades, industries faced the problem of how to connect devices to speak together with minimum wiring. Philips Semiconductors (now NXP Semiconductors) faced this problem when needing to connect many integrated circuits (ICs) together, and Bosch, in the automotive industry, faced similar problems when connecting tens of microcontrollers in each car. Recently, with smart homes, the problem started to be part of every home: you may want to build your smart home with accessories from different manufacturers and you want these devices to speak together, that is, to communicate with each other. This problem is compounded when you consider central control through an app or a voice assistant.

Philips found the solution in late 1970s and introduced the Inter-integrated Circuits (I2C) network protocol. I2C was designed as a method to provide an interface between microprocessors and peripheral devices (like keyboard, mouse, memory, and IO/serial/parallel ports) without wiring full address, data, and control busses between devices.

Bosch solved the problem in 1986 by introducing Controller Area Network (CAN) protocol as a multi-master, message broadcast robust vehicle bus standard that allows microcontrollers and devices to communicate with each other in applications without a host computer. I2C and CAN are wired communication standards.

Concerning home automation, a number of common home automation protocols that allow different devices to speak and communicate together appeared during the last few decades. This is started by “Universal Powerline Bus (UPB)” and reached to Z-Wave protocol and now Thread. Some of the most popular protocols in use today include the following:

- Ethernet
- Wi-Fi
- WiMax
- Z-Wave
- ZigBee
- Bluetooth

- Thread
- Insteon
- X10
- Universal Powerline Bus (UPB).

Each technology has its own benefits and limitations.

Wi-Fi, WiMax and Insteon were part of our book “Serial Communication Protocols and Standards”. This book introduces to the reader comprehensive knowledge on the following protocols: I2C (IIC), I3C, CAN, ZigBee, Bluetooth, and Z-Wave. To complete the knowledge of the reader, we give a short summary for the remaining protocols: Ethernet, Thread, Insteon, X10, and UPB.

The book is divided into sections; each consists of a number of chapters. Each section covers one technology.

Section 1: Inter-integrated Circuits (IIC)

This section covers IIC technology in three chapters:

- Chapter 1: Inter-integrated Circuits (IIC)
- Chapter 2: I2C Design and Operation
- Chapter 3: Improved Inter-integrated Circuits (I3C)

Section 2: Controller Area Network (CAN)

This section covers Controller Area Network protocol in two chapters:

- Chapter 4: Controller Area Network (CAN)
- Chapter 5: CAN Data Link Layer

Section 3: ZigBee

This section covers ZigBee protocol in three chapters:

- Chapter 6: IEEE 802.15.4 Standard and ZigBee
- Chapter 7: ZigBee Stack Layers
- Chapter 8: ZigBee Pro and ZigBee Security

Section 4: Bluetooth

This section covers Bluetooth technology in one chapter:

Chapter 9: Bluetooth

Section 5: Z-Wave

This section covers Z-Wave technology in one chapter:

Chapter 10: Z-Wave Technology

Section 6: Comparisons

The last chapter of the book gives a comprehensive comparison between the different smart home protocols:

Chapter 11: Smart Home Protocols: Comparison



Taylor & Francis

Taylor & Francis Group

<http://taylorandfrancis.com>

Acknowledgment

The authors of this book would like to express their special thanks of gratitude to Microchip for giving them the permission and the opportunity to use materials that came from Microchip scientific notes.



Taylor & Francis

Taylor & Francis Group

<http://taylorandfrancis.com>

List of Figures

Figure 1.1	Example of I2C bus applications.	5
Figure 1.2	The I2C protocol format.	7
Figure 1.3	Data transfer timing diagram.	7
Figure 1.4	I2C hardware.	8
Figure 1.5	I2C bus and interface hardware: Case of connect- ing two devices with the same supply voltage (www.freescale.com).	10
Figure 1.6	Connecting two devices with different supply voltage.	11
Figure 1.7	Basic configuration of I2C.	13
Figure 1.8	Application example showing I2C communication between the different ICs on a system and with pull- up resistors on I2C bus.	14
Figure 1.9	I2C data validity.	18
Figure 1.10	Start and stop conditions.	18
Figure 1.11	7-bit address.	19
Figure 1.12	I2C device addressing.	20
Figure 1.13	7-bit addressing structure.	20
Figure 1.14	I2C acknowledgment scheme (Assumption: Master is the transmitter).	21
Figure 1.15	Data transfer on the I2C bus.	21
Figure 1.16	10-bit addressing.	22
Figure 1.17	10-bit addressing structure.	23
Figure 1.18	A master-transmitter addresses a slave-receiver with a 10-bit address.	24
Figure 1.19	A master-receiver addresses a slave-transmitter with a 10-bit address.	24
Figure 1.20	General call address format.	26
Figure 1.21	Data transfer from a hardware master-transmitter. .	27
Figure 1.22	Data transfer by a hardware-transmitter capable of dumping data directly to slave devices.	28

Figure 1.23	START byte procedure.	29
Figure 1.24	Device ID field.	29
Figure 1.25	Master-to-slave data transfer.	32
Figure 1.26	Slave-to-master data transfer.	33
Figure 1.27	Read event.	34
Figure 1.28	Bidirectional data transfer.	35
Figure 1.29	Read/write cycle.	37
Figure 1.30	Acknowledge extension (a) without and (b) with extension.	37
Figure 1.31	Multiple address.	39
Figure 1.32	I2C bus configuration with HS devices.	43
Figure 1.33	Details of the first part of high speed transfer format.	44
Figure 1.34	Data transfer format in HS-mode.	44
Figure 1.35	A complete HS-mode transfer.	45
Figure 1.36	I2C bus interface—multiple master.	47
Figure 1.37	Bus arbitration.	48
Figure 1.38	Clock synchronization.	50
Figure 1.39	Clock synchronization during the arbitration procedure.	52
Figure 2.1	Basic internal structure of SDA/SCL lines.	56
Figure 2.2	I2C bus interface—a closer look (Image source infoindustrielle.free.fr).	56
Figure 2.3	Pulling the bus low with an open-drain interface.	57
Figure 2.4	Releasing the bus with an open-drain interface.	58
Figure 2.5	Specified voltage levels for logical high and low.	59
Figure 2.6	Open-collector topology and equivalent circuit. (R_{DSON} is the drain to source resistance when the transistor is ON.)	59
Figure 2.7	Charge time for transition between logical low to high.	60
Figure 2.8	Minimum pull-up resistance [R_{P} (min)] versus pull-up reference voltage (V_{CC}).	62
Figure 2.9	Maximum pull-up resistance [R_{P} (max)] versus bus capacitance (C_{b}).	62
Figure 2.10	Total high-level input current as a function of the maximum value of R_{p} with supply voltage as a parameter.	64

Figure 2.11	Timing diagram for an I2C system with the master generating the SCL clock signal and a slave responding with data on SDA.	66
Figure 2.12	Generic I2C bus system showing a bus master, repeater, and a slave.	67
Figure 2.13	Timing diagram for an I2C system with the master generating the SCL clock signal, an I2C repeater providing capacitance buffering (and adding propagation delay), and a slave responding with data on SDA.	68
Figure 2.14	Series resistor (R_s) for protection against high-voltage spikes.	71
Figure 2.15	Maximum value of R_s as a function of the value of R_p with supply voltage as a parameter. The value of the series resistance can be defined practically based on the value of R_p used.	72
Figure 2.16	Simplified equivalent circuit diagram for an I2C connection between two devices (master or slave) containing all relevant factors for I2C.	72
Figure 2.17	SDA (a) and SCL (b) as measured at I2C device #1 (master).	73
Figure 2.18	SDA (a) and SCL (b) as measured at I2C device #2 (slave).	73
Figure 2.19	SDA and SCL measured at I2C device #2 (slave) with increased R_p	73
Figure 2.20	A complete master receiver transfer between an I2C master and I2C slave device with $R_s = 200 \Omega$, measured at the slave device.	74
Figure 2.21	Effect of C_p and R_p (Images from: Tracii XL 2.0).	75
Figure 2.22	Crosstalk between SDA and SCL.	76
Figure 2.23	Example of using Bidirectional drivers to reduce the effects of WIRING capacitance.	79
Figure 2.24	Schematic for SCL/SDA delay generation to be fitted at the master.	82
Figure 2.25	Schematic for SDA delay generation fitted at each slave.	82
Figure 2.26	I2C bus utilizing I2C buffer, which allows more slave devices.	83
Figure 2.27	Use of multiplexor/switches and repeater.	84

Figure 2.28	Switched pull-up circuit.	85
Figure 2.29	Operation of an I2C voltage translator/level shifter.	86
Figure 2.30	Slave address conflict solved with I2C multiplexor.	88
Figure 2.31	I2C bus with multiple slave devices with the same address.	88
Figure 2.32	Eight-channel I2C switch example.	89
Figure 2.33	AVR I2C.	98
Figure 2.34	Circuit diagram.	104
Figure 3.1	Typical application processor or sensor hub system with various digital interfaces.	115
Figure 3.2	SMBus ARP commands.	122
Figure 3.3	Dynamic address assignment in I3C.	122
Figure 3.4	I2C system with external sideband GPIO signals.	126
Figure 3.5	I3C system with in-band interrupt support.	126
Figure 3.6	Energy consumption and raw data rate: I3C versus I2C (figure courtesy from https://www.synopsys.com/dw/doc.php/wp/mipi_i3c_wp.pdf).	128
Figure 4.1	Typical automotive networks.	139
Figure 4.2	The OSI 7-layer model and the ISO 11898 standard.	145
Figure 4.3	CAN layers and HLP.	145
Figure 4.4	The layered ISO 11898 standard architecture and its reflection on CAN node requirements.	146
Figure 4.5	The four abstraction levels of CAN protocol.	146
Figure 4.6	Broadcast communication.	151
Figure 4.7	CAN bus (ISO-11898-2).	151
Figure 4.8	Differential signal: Advantage of using differential balanced 2-wire interface in noise elimination.	152
Figure 4.9	Basic CAN bus connection.	153
Figure 4.10	CAN bus electrical interface circuit.	153
Figure 4.11	The inverted logic of a CAN bus.	154
Figure 4.12	ISO11898 nominal bus levels.	155
Figure 4.13	One-way propagation delay.	158
Figure 4.14	Termination converts.	160
Figure 4.15	CANopen DSUB connector.	161
Figure 4.16	5-Pin mini-connector (ANSI/B.93.55M-1981).	161
Figure 4.17	CAN node.	161
Figure 4.18	CAN node types.	163
Figure 4.19	Bit timing.	166
Figure 4.20	Bit segments.	168

Figure 4.21	Bit-timing for CAN FD.	169
Figure 4.22	TQ and the bit period.	170
Figure 4.23	CAN bit control registers (For MCP2510 CNF Register).	172
Figure 4.24	Bit propagation delay.	174
Figure 4.25	One-way propagation delay.	174
Figure 4.26	Sample point of CAN bit.	176
Figure 4.27	Synchronizing the bit time.	178
Figure 4.28	Maximum time between synchronization edges. . .	180
Figure 4.29	Bit timing for maximum oscillator tolerance. . . .	182
Figure 4.30	Bit-timing for maximum bit rate.	183
Figure 4.31	Pin description of 9-pin connector.	184
Figure 4.32	V-CAN transceiver power savings.	186
Figure 4.33	Common-mode noise coupled onto 4 twisted-pair bus lines.	187
Figure 4.34	Split termination.	189
Figure 4.35	MCP2551 block diagram.	191
Figure 4.36	TXD permanent dominant detection.	192
Figure 4.37	Power-on reset and brown-out detection.	193
Figure 4.38	Receiving (node ground) below transmitting (node ground).	194
Figure 4.39	Receiving (node ground) above transmitting (node ground).	195
Figure 5.1	CAN data frame.	198
Figure 5.2	CAN communication services.	199
Figure 5.3	CAN broadcast communication.	199
Figure 5.4	Remote transmission request (RTR).	200
Figure 5.5	Multiple bus access.	201
Figure 5.6	Bus arbitration method.	202
Figure 5.7	CSMA/CD-CR example: Case of two nodes. . . .	204
Figure 5.8	Synchronization and re-synchronization.	205
Figure 5.9	Arbitration field in case of standard and extended formats.	206
Figure 5.10	CAN frame.	206
Figure 5.11	Base data frame format.	207
Figure 5.12	Standard CAN—The 11-bit identifier.	207
Figure 5.13	Control field: DLC field (d, dominant; r, recessive). .	208
Figure 5.14	Data field.	209
Figure 5.15	CRC field.	209

Figure 5.16	Acknowledge field.	211
Figure 5.17	End of frame (EOF).	211
Figure 5.18	Extended CAN: 29-bit identifier.	212
Figure 5.19	Data frame format.	214
Figure 5.20	Remote data frame.	217
Figure 5.21	Error frame.	218
Figure 5.22	Overload frame.	220
Figure 5.23	Bit stuffing rule.	221
Figure 5.24	Complete frame before and after stuffing.	222
Figure 5.25	Form error.	224
Figure 5.26	CRC error.	225
Figure 5.27	Acknowledge error.	226
Figure 5.28	Stuff error.	226
Figure 5.29	Bit error.	227
Figure 5.30	Local errors in EOF.	228
Figure 5.31	Error states of CAN node.	229
Figure 5.32	Passive error frame.	230
Figure 5.33	Suspend passive transmission.	231
Figure 5.34	Transmit error counter.	231
Figure 5.35	Receive error counter.	232
Figure 5.36	Example of error confinement.	236
Figure 5.37	CAN FD format.	240
Figure 5.38	CAN FD.	240
Figure 5.39	CAN FD frame.	241
Figure 5.40	FD base and extended frame format.	241
Figure 5.41	FBFF and FEFF.	242
Figure 5.42	CRC fields.	243
Figure 5.43	Global time producer.	244
Figure 5.44	CAN node.	245
Figure 5.45	CAN message.	246
Figure 5.46	CAN receiving message.	246
Figure 6.1	Wireless technologies compared.	258
Figure 6.2	Architecture of ZigBee node.	261
Figure 6.3	Types of network nodes in ZigBee architecture (ZigBee stack) (https://mmbnetworks.atlassian.net/wiki/spaces/SKB/pages/39518242/Zigbee+Network+Concepts).	262
Figure 6.4	Structure of ZigBee and IEEE 802.15.4 architecture.	269

Figure 6.5	Outline of the ZigBee stack architecture (https://research.kudelskisecurity.com/2017/11/01/zigbee-security-basics-part-1/).	270
Figure 6.6	APS sublayer model.	273
Figure 6.7	APS frame format.	274
Figure 6.8	APS Ack frame format.	274
Figure 6.9	APS command frame.	274
Figure 6.10	Network layer frame format.	276
Figure 6.11	APL layer security framework.	279
Figure 6.12	Network of Example 6.1.	280
Figure 6.13	The network with the addresses.	281
Figure 6.14	Star topology.	284
Figure 6.15	Mesh (peer-to-peer) topology.	285
Figure 6.16	Tree topology.	287
Figure 6.17	Formation of cluster tree network.	288
Figure 6.18	Cluster tree topology.	289
Figure 6.19	WPAN architecture.	290
Figure 6.20	Channelization and spectrum spacing.	295
Figure 6.21	PPDU format.	297
Figure 7.1	MAC sublayer.	302
Figure 7.2	Superframe structure.	302
Figure 7.3	Superframe format: Details of the active period of the superframe.	303
Figure 7.4	If the BLE option is selected, the coordinator receiver is active only for a limited time after the IFS period.	305
Figure 7.5	Communication to a coordinator (a) in a beacon-enabled network and (b) in a nonbeacon-enabled network.	307
Figure 7.6	Communication from a coordinator to device (a) in a beacon-enabled network and (b) in a nonbeacon-enabled network.	308
Figure 7.7	Network formation sequence diagram.	310
Figure 7.8	Beacon NWK payload format.	313
Figure 7.9	NWK discovery sequence diagram.	314
Figure 7.10	NWK join sequence diagram—child side.	316
Figure 7.11	NWK sequence diagram—parent side.	317
Figure 7.12	Device association.	318
Figure 7.13	Device disassociation.	319

Figure 7.14	The sequence chart for orphan device notification. . .	320
Figure 7.15	MAC frame.	325
Figure 7.16	MAC frame with detailed frame control field. . . .	326
Figure 7.17	A simple modulo 2 division example.	328
Figure 7.18	MAC inter-frame space.	329
Figure 7.19	Data frame structure.	330
Figure 7.20	Beacon frame format.	330
Figure 7.21	Acknowledgment frame structure.	332
Figure 7.22	MAC command frame structure.	333
Figure 7.23	MAC command frame format.	333
Figure 7.24	RREQ.	341
Figure 7.25	RREP.	342
Figure 7.26	Sample route request (broadcast) transmission where R3 is trying to discover a route to R6.	342
Figure 7.27	Destination node received the rout request.	343
Figure 7.28	Node 1 requesting rout to node 10.	344
Figure 7.29	Multicast discovery.	346
Figure 7.30	Node 1 requesting route to node 10.	348
Figure 7.31	Tree of the example.	351
Figure 7.32	Example 2 on hierarchy routing.	352
Figure 7.33	Many-to-one routing.	356
Figure 7.34	RF4CE multi-star topology.	357
Figure 8.1	In larger networks, many routes to a centralized mode may cause routing table overflows near the central node.	364
Figure 8.2	Through shared many-to-one routes and source routing, less routing table space is used.	365
Figure 8.3	Asymmetric link.	367
Figure 8.4	IEEE MAC frame.	371
Figure 8.5	MAC frame: Auxiliary security frame.	372
Figure 8.6	Data payload field.	374
Figure 8.7	The two ZigBee security models: Centralized and distributed.	377
Figure 8.8	ZigBee frame with security at the NWK layer. . . .	380
Figure 8.9	ZigBee frame with security at the APS layer. . . .	381
Figure 8.10	ZigBee frame with security at the MAC layer. . . .	381
Figure 8.11	ZigBee security commercial mode.	385
Figure 8.12	ZigBee security residential mode.	386

Figure 8.13	ZigBee security key summary for centralized model.	387
Figure 8.14	Role of AES-CCM* in data authentication and confidentiality.	389
Figure 8.15	Security problems start with pre-configured link key.	400
Figure 8.16	ZigBee 3.0 security starting with install code.	401
Figure 8.17	Applications of ZigBee technology.	407
Figure 9.1	Two products communicating over a wireless Bluetooth radio link.	416
Figure 9.2	Bluetooth cards.	416
Figure 9.3	Piconets and scatternets.	418
Figure 9.4	Mapping classic Bluetooth (IEEE 802.11.5 v1.1) and IEEE 802.11 (Wi-Fi) to the data link and physical layers of the OSI model.	423
Figure 9.5	Bluetooth protocol stack.	424
Figure 9.6	Bluetooth protocol stack (Bluetooth 4).	425
Figure 9.7	Packet lengths supported by baseband.	428
Figure 9.8	Bluetooth components involved in service discovery.	434
Figure 9.9	State transitions during the inquiry process.	436
Figure 9.10	Timing of the inquiring device's behavior.	436
Figure 9.11	Periodic inquiry and inquiry scan.	437
Figure 9.12	State transitions during the page process.	439
Figure 9.13	Page procedure.	441
Figure 9.14	Unlike an inquiry, a service search is only between two devices.	442
Figure 9.15	Example of a data element containing a 16-bit unsigned int.	444
Figure 9.16	SDP session messaging.	445
Figure 9.17	The UPnP protocol stack.	446
Figure 9.18	For two devices to complete the pairing process, a common PIN must be entered.	450
Figure 9.19	Trigger points are located within RFCOMM and L2CAP to invoke mode 2 security.	453
Figure 9.20	Link-key generation with PIN [19].	456
Figure 9.21	Link-key establishment for SSP.	456
Figure 9.22	MAC spoofing attack.	463
Figure 9.23	PIN cracking attack.	464
Figure 9.24	Man-in-the-middle attack.	465
Figure 9.25	BlueSnarfing attack.	466

Figure 10.1	Z-Wave protocol stack.	483
Figure 10.2	Z-Wave network.	485
Figure 10.3	Z-Wave frames.	491
Figure 10.4	Z-Wave layer transmitter.	492
Figure 10.5	MAC frame.	493
Figure 10.6	Manchester coding.	494
Figure 10.7	MAC layer frame types.	494
Figure 10.8	Collision avoidance zone.	495
Figure 10.9	Z-Wave basic frame format.	497
Figure 10.10	Singlecast.	498
Figure 10.11	Case of multicast.	499
Figure 10.12	Case of broadcast.	500
Figure 10.13	Z-Wave frame format in different layers.	500
Figure 10.14	Routed singlecast.	501
Figure 10.15	Routed acknowledgment.	501
Figure 10.16	Network topology and routing table.	502
Figure 10.17	Routing in a Z-Wave network.	502
Figure 10.18	Route between controller and node 4 (a) through node 3, (b) through node 2.	503
Figure 10.19	Alternative Z-Wave network routing.	504
Figure 10.20	Z-Wave application frame format.	504
Figure 10.21	Get node info frame flow.	506
Figure 10.22	Z-Wave key exchange protocol.	510
Figure 11.1	IoT system (Image from “International Journal on Future Revolution in Computer Science & Com- munication Engineering ISSN: 2454–4248 Volume: 3I”).	527
Figure 11.2	Thread protocol stack.	531
Figure 11.3	Network layer comparison.	533
Figure 11.4	Network security comparison.	533

List of Tables

Table 1.1	Reserved addresses (X = don't care; 1 = high; 0 = low)	26
Table 1.2	Assigned manufacturer IDs	31
Table 1.3	Non-reserved addresses in the 7-bit address space . .	31
Table 1.4	Applicability of I2C bus protocol features	40
Table 2.1	Parametric from I2C specifications	61
Table 2.2	f_{SCL} (max) calculation for I2C systems based on FM or FM+ masters and slaves	66
Table 2.3	Relationship between t_{LOW} (min), $t_{VD;DAT}$, and $t_{SU;DAT}$	67
Table 2.4	Maximum clock frequency (f_{SCL} (max)) calculation based on I2C FM specifications and for FM specifications with TCA9617B repeater characteristics	69
Table 2.5	Maximum clock frequency (f_{SCL} (max)) calculation based on I2C FM+ specifications and for FM+ specifications with TCA9617B repeater characteristics	70
Table 4.1	Maximum bit rate versus bus length	156
Table 4.2	CAN bit rate versus bus length	179
Table 5.1	Standard CAN frame	212
Table 5.2	Extended CAN format	213
Table 5.3	CAN 2.0A message frame	215
Table 5.4	CAN 2.0B message frame (extended frame format) .	216
Table 5.5	Bus loading example	238
Table 6.1	ZigBee, Bluetooth, and Wi-Fi characteristics	260
Table 6.2	IEEE 802.15.4 standard summary	283
Table 6.3	Physical layer frequency band	293
Table 6.4	IEEE 802.15.4 RF channel details	293
Table 6.5	Summary of the frequency bands and data rates of ZigBee PHY version	293
Table 6.6	ZigBee channels, 915 MHz ISM band	294
Table 6.7	ZigBee channels, 2.4 GHz ISM band	295

Table 6.8	Coding in case of 2.4 GHz	296
Table 6.9	Generic packet format	297
Table 6.10	Preamble field lengths and durations	298
Table 6.11	SFD field format (except for ASK PHYs)	298
Table 6.12	SFD field lengths	298
Table 6.13	Frame length values	299
Table 7.1	Frame control field	326
Table 7.2	Frame type subfield	327
Table 7.3	MAC command frames	334
Table 7.4	Broadcast transaction table (BTT)	339
Table 7.5	Route table when using DSR	349
Table 7.6	Neighbor table entry—mandatory	355
Table 7.7	Neighbor table entry—optional	356
Table 8.1	Comparison of ZigBee and ZigBee PRO feature sets	360
Table 8.2	ZigBee versions' features	361
Table 8.3	Comparison of 2006, ZigBee, and ZigBee PRO feature sets	369
Table 8.4	ZigBee security levels	373
Table 9.1	Examples of bluetooth profiles	427
Table 9.2	Relationship between scan interval, train repetition and paging modes	440
Table 9.3	Associated protocol service multiplexor specifying the service it represents	453
Table 9.4	Bluetooth threat taxonomy	461
Table 10.1	Z-Wave frequency bands	481
Table 10.2	Z-Wave features	482
Table 10.3	Properties of the Z-Wave device models	488
Table 10.4	Typical applications for slaves	488
Table 10.5	Home ID and node ID comparison	490
Table 10.6	Z-Wave data rate and accuracy	492
Table 10.7	Modulation and coding format in Z-Wave physical layer	493
Table 10.8	Routing table for the Z-Wave network	503
Table 11.1	ZigBee and Z-Wave specifications and capabilities .	514
Table 11.2	Z-Wave versus ZigBee versus INSTEON comparison table	525

List of Abbreviations

ACK	Acknowledge
AES	Advanced Encryption Standard
AODV	On-demand Distance Vector
AODVjr	On-demand Distance Vector Junior
APL	Application Layer
APS	Application Support Sublayer
ARP	Address Resolution Protocol
BOD	Brown-Out Detection
CAN	Controller Area Network
CCC	Common Command Code
CD	Collision Detection
CD	Coordinator Device
CoAP	Constrained Application Protocol
CR	Collision Resolution
CRC	Cyclic Redundancy Check
CSMA/CD	Carrier Sense Multiple Access with Collision Detection
CSMA/CD +AMP	CSMA/CD and Arbitration on Message Priority
CTS	Conformance Test Suite
DDA	Dynamic Address Assignment
DDC	Display Data Channel
DDoS	Distributed Denial-of-Service (attack)
DLC	Data Length Code
DoS	Denial-of-Service
DSR	Dynamic Source Routing
DTLS	Datagram Transport Layer Security
ED	End Device
EMI	Electromagnetic Interface
EOF	End of Frame
ESD	Electrostatic Discharge
ESI	Electrostatic Interface

FFD	Fully Functional Device
Fm	Fast Mode
Fm+	Fast-mode Plus
GAP	Generic Access Profile
GFSK	Gaussian Frequency Shift Keying
HAN	Home-Area Networks
HCI	Host Controller Interface
HDR	High Data Rate (a set of I3C bus modes)
HDR-DDR	HDR Double Data Rate (an I3C bus mode)
HDR-TSP	HDR Ternary Symbol for Pure Bus (an I3C bus mode)
HLP	Higher Layer Protocol
Hs	High-speed Mode
HSP	Headset Profile
IBHR	In-band Hardware Reset (an expected I3C bus feature)
IBI	In-band Interrupt (an I3C bus feature)
IIC	Inter-integrated Circuit
IID	Interface Identifier
IIIC	Improved Inter-integrated Circuit
IFS	Inter Frame Space
IPMI	Intelligent Platform Management Interface
IPT	Information Processing Time
ISM	Industrial, Scientific, and Medical (Radio frequency)
KEK	Key Encryption Key
L2CAP	Logical Link Control and Adaptation Protocol
LLL	Logical Link Control
LMP	Link Manager Protocol
LR-WPAN	Low Rate Wireless Personal Area Network
LQI	Link Quality Indication
MAC	Medium Access Control (Media access control)
MAC	Message Authentication Code
MANET	Mobile Ad Hoc Network
MIC	Message Integrity Code
MQTT	Message Queuing Telemetry Transport
NIB	Network Layer Information Base
NLME	Network Layer Management Entity
NRZ	Non-return to Zero
NWK	Network Layer
OBEX	Object Exchange Protocol
ODR	Output Data Rate

PAN	Personal Area Network
PHY	Physical Layer
PLC	Power-Line Communications
PMBus	Power Management Bus
POR	Power-on Reset
PPDU	PHY Protocol Data Unit
PPP	Point-to-point Protocol
PSDU	PHY Service Data Unit
RFCOMM	Radio Frequency Communication
RFD	Reduced Functional Device
RTR	Remote Transmission Request
SCL	Serial Clock Line
SDA	Serial Data (an I3C bus line)
SDDb	Service Discovery Database
SDR	Single Data Rate
SDL	Serial Data Line
SDAP	Service Discovery Application Protocol
SDP	Service Discovery Protocol
SDR	Single Data Rate (an I3C bus mode)
SFD	Start-of-frame Delimiter
SJW	Synchronous Jump Width
SKKE	Symmetric-Key Key Establishment protocol
SMBus	System Management Bus
SOF	Start of Frame
SPI	Serial Peripheral Interface (an interface specification)
SPP	Serial Port Profile
SRR	Substitute Remote Request
STP	Shielded Twisted Pair
SUC	Static Update Controller
TDC	Transmitter Delay Compensation
TTC	Time-triggered Communication
TTCAN	Time-triggered CAN
TWI	Two-wire Interface
UFm	Ultra-fast Mode
UPB	Universal Powerline Bus
UTP	Un-shielded Twisted Pair
WPAN	Wireless Personal Area Network
ZDO	ZigBee Device Objects
ZiCL	ZigBee Cluster Label
ZTC	ZigBee Trust Center



Taylor & Francis

Taylor & Francis Group

<http://taylorandfrancis.com>

Section I

Inter-integrated Circuits (IIC)

This section deals with IIC. It consists of three chapters:

Chapter 1: Inter-integrated Circuits (IIC)

Chapter 2: IIC Design and Operation

Chapter 3: Improved Inter-integrated Circuits (I3C)

Some of the topics covered in this section are:

- I2C protocol
- The I2C bus hardware structure
- Electrical consideration
- The physical bus–I2C bus interface
- Masters and slaves
- I2C data validity
- Voltage levels and resistor values
- Addressing–addressing standards
- I2C bus transaction
- Clock stretching
- Applicability of I2C bus features
- I2C modes: bus speeds
- I2C as a multi-master bus: bus arbitration
- Clock synchronization and handshaking
- I2C interface: connecting I2C bus to a PC
- Design of I2C bus
- Calculation of the pull-up resistor
- Maximum clock frequency of I2C bus
- Use of repeaters

- Series protection resistors
- Managing the delays in large systems
- Use of multiplexor/switches to solve address conflict
- Common problems in I2C bus systems
- Obscure problems in systems
- Typical problems on the I2C bus
- Derivative technologies
- Improved inter-integrated circuits (I3C)
- I3C bus configuration and device roles
- I3C features
- Dynamic addressing
- I3C bus activity states
- Error detection and recovery methods in I3C
- Resolving communication conflicts on the I3C bus
- MIPI I3C specifications and software development

1

Inter-integrated Circuits (IIC/I2C)

1.1 Introduction

The most popular form of serial bus protocol is the “Inter-integrated Circuit” (IIC). Sometimes, the bus is called I2C bus (pronounced eye-to-see) or I²C (pronounced eye-squared see). I2C was developed as a communication protocol to interact between different ICs on a motherboard, a simple internal bus system. This standard was originally developed by Philips Semiconductors (now NXP Semiconductors) in the late 1970s as a method to provide an interface between microprocessors and peripheral devices (like keyboard, mouse, memory, and IO/serial/parallel ports) without wiring full address, data, and control busses between devices; more specifically, it was designed to provide an easy way to connect a CPU to peripheral chips in a TV set. The problem faced by the designers at that time until now is that the peripheral devices in embedded systems are often connected to the microcomputer unit (normally a microcontroller) as memory mapped I/O devices, using the microcontroller’s parallel address and data bus. This results in lots of wiring on the printed circuit boards (PCBs) to route the address and data lines. The number of wires increases if we consider the need to connect also a number of address decoders and glue logic to connect everything. In mass production items such as TV sets, videocassette recorders (VCRs), and audio equipment, this is not acceptable. In such applications, every component that can be saved means increased profitability for the manufacturer and more affordable products for the end customer. Another problem directly related to the use of a lot of wires is the possible interference: lots of control lines imply that the system is more susceptible to disturbances by electromagnetic interference (EMI) and electrostatic discharge (ESD). The need for a more effective and economical way of connecting different devices (normally ICs) resulted in introducing the 2-wire communication bus that we call it now as

4 Inter-integrated Circuits (IIC/I2C)

I2C bus protocol. This protocol enables peripheral ICs in electronic systems to communicate with each other using simple communication hardware. The ICs can be on the same board or linked via cables. The length of the cable is limited by the total bus capacitance and the noise generated on the bus.

Today, I2C has become a *de facto* world standard that is now implemented in over 1000 different ICs and is licensed to more than 50 companies including some of the leading chip manufacturers like Xicor, ST Microelectronics, Infineon Technologies, Intel, Texas Instruments, Maxim, Atmel, Analog Devices, and others.

Additionally, the versatile I2C bus is used in various control architectures such as System Management Bus (SMBus), Power Management Bus (PMBus), Intelligent Platform Management Interface (IPMI), Display Data Channel (DDC), and Advanced Telecom Computing Architecture (ATCA). Figure 1.1 shows an example of I2C bus applications.

Concerning SMBus, it is defined by Intel in 1995 and is a subset of I2C, defining a stricter usage. One purpose of SMBus is to promote robustness and interoperability. Accordingly, modern I2C systems incorporate some policies and rules from SMBus, sometimes supporting both I2C and SMBus, requiring only minimal reconfiguration either by commanding or by output pin use.

The **Power Management Bus** (PMBus) is a variant of the SMBus which is targeted at digital management of power supplies. Like SMBus, it is a relatively slow-speed two-wire communication protocol based on I2C. Unlike either of those standards, it defines a substantial number of domain-specific commands rather than just saying how to communicate using commands defined by the reader.

Since October 10, 2006, no licensing fees are required to implement the I2C protocol. However, fees are required to obtain I³C slave addresses allocated by NXP.

1.1.1 I2C Revisions

The history of I2C specification releases:

- In 1982, the original 100 kHz I2C system was created as a simple internal bus system for building control electronics with various Philips chips.
- In 1992, Version 1 added 400 kHz *Fast mode* (*Fm*) and a 10-bit addressing mode to increase capacity to 1008 nodes. This was the first standardized version.

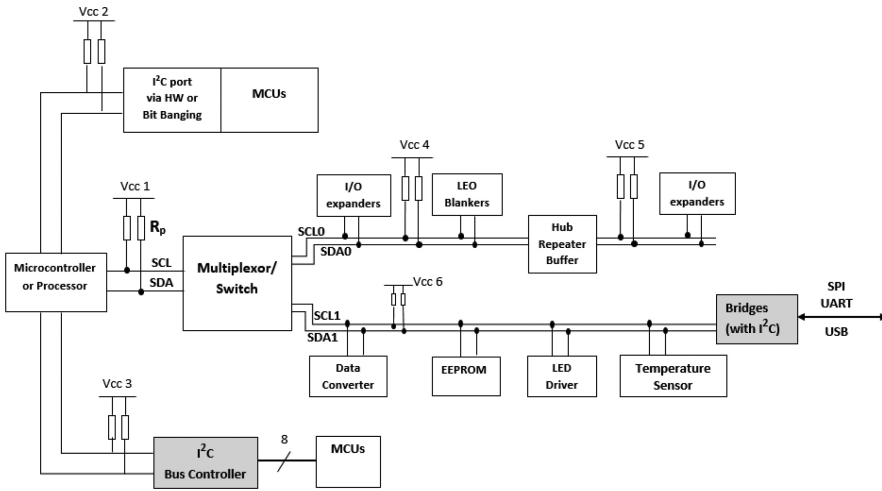


Figure 1.1 Example of I2C bus applications.

- In 1998, Version 2 added 3.4 MHz *High-speed mode (Hs)* with power-saving requirements for electric voltage and current.
- In 2000, Version 2.1 clarified version 2, without significant functional changes.
- In 2007, Version 3 added 1 MHz *Fast mode plus (Fm+)* (using 20 mA drivers), and a device ID mechanism.
- In 2012, Version 4 added 5 MHz *Ultra-fast mode (UFm)* for new USDA (data) and USCL (clock) lines using push-pull logic without pull-up resistors, and added an assigned manufacturer ID table. It is only a unidirectional bus.
- In 2012, Version 5 corrected mistakes.
- In 2014, Version 6 corrected two graphs. This is the most recent standard.

This chapter helps the reader to understand this very popular protocol. The knowledge given in the chapter will help device and system designers to understand how the I2C bus works and to help them to implement a working application. Various operating modes are described. It contains a comprehensive introduction to the I2C bus data transfer, handshaking, and bus arbitration schemes. Detailed sections cover the timing and electrical specifications for the I2C bus in each of its operating modes.

The main reference of this chapter is the NXP Semiconductor (former Philips Semiconductors) note UM10204 I2C bus specification and user

manual. Other references are used and given in the “Reference” section of the chapter.

1.2 I2C Bus Terminology

Transmitter: The device which sends the data or message to the receiver.

Receiver: The device which receives the data or message from the bus.

Master: The device which generates clock signals, initiates a transfer, that is, start condition, and terminates a transfer, that is, stop condition. It may also act as a transmitter/receiver.

Slave: The device addressed by a master. Also it may act as a transmitter/receiver.

Multi-master: More than one master can attempt to control the bus at the same time without demeaning the data/message.

Arbitration: It is a procedure to ensure that if more than one master simultaneously try to control the bus, only one is allowed to do so and the data are not corrupted.

Synchronization: It is a procedure to synchronize the clock signals of two or more devices.

Serial data line (SDA): The **serial data line (SDA)** is the line on which all the data transfer among the devices takes place.

Serial clock line (SCL): The **serial clock line (SCL)** is the serial clock. I2C is a synchronous protocol, and hence, SCL is used to synchronize all the devices and the data transfer together. We'll learn how it works a little later in this post.

1.3 I2C Protocol

The I2C protocol is a very simple protocol. The protocol describes the format of a message, while communication between ICs connected to the bus. Also it gives the timing diagram of an entire transaction—from start to stop.

1.3.1 Transaction Format–Message Format

An I2C transaction consists of one or more messages. Each message (Figure 1.2) begins with a start symbol, and the transaction ends with a stop

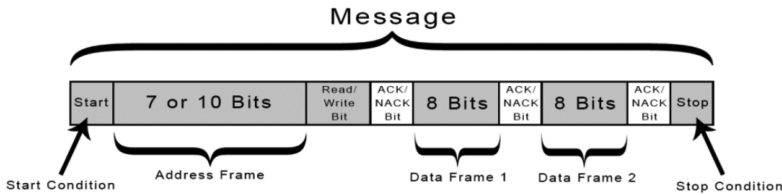


Figure 1.2 The I2C protocol format.

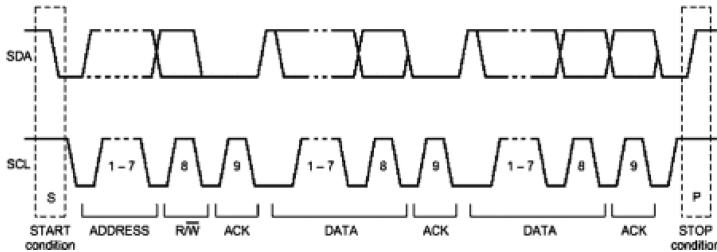


Figure 1.3 Data transfer timing diagram.

symbol. Start symbols after the first, which begin a message but not a transaction, are referred to as repeated start symbols.

Each message is a read or a write. A transaction consisting of a single message is called either a read or a write transaction. A transaction consisting of multiple messages is called a combined transaction. The most common form of the latter is a write message providing intra-device address information, followed by a read message.

Many I2C devices do not distinguish between a combined transaction and the same messages sent as separate transactions, but not all. The device ID protocol requires a single transaction; slaves are forbidden from responding if they observe a stop symbol. Configuration, calibration, or self-test modes which cause the slave to respond unusually are also often automatically terminated at the end of a transaction.

1.3.2 Timing Diagram

Figure 1.3 shows the real shape of the signals while changing state from high to low and from low to high. Ideally, whenever a signal changes its state (from high to low or vice versa), it is supposed to do so immediately. But in real scenario, it is almost impossible for it to happen without a time lag. This time lag is shown by the skew lines. The protocol is as follows:

- The transaction starts with a START sequence. During the start condition, the serial data line (SDA) switches from a high-voltage level to a

8 Inter-integrated Circuits (IIC/I2C)

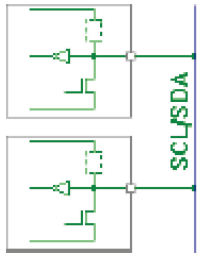


Figure 1.4 I2C hardware.

low-voltage level before the serial clock line (SCL) switches from high to low (see Figure 1.4 for SDA and SCL and also Section 1.6).

- After the START sequence, the *Master* has to send the address of the *Slave* it wants to talk to, that is, the 7-bit ADDRESS (MSB first) followed by R/W' bit determining whether it is required to read from the device or write into it. If the master wants to send data to the slave, the read/write bit is a low-voltage level. If the master is requesting data from the slave, the bit is a high-voltage level
- The *Slave* responds by acknowledging the address using ACK/NACK bit. In case if there is no ACK, the *Master* will act as mentioned before.
- Once the *Slave* acknowledges the address, it means that it is now ready to send/receive data to/from the *Master*. Thus begins the data transfer. The DATA are always of 8 bits (MSB first), and the receiver has to send the ACK signal after each byte of data received.
- When the transaction is over, the *Master* ends it by generating a STOP sequence. Alternatively, *Master* could also begin with a repeated START.

The above describes the standard format. Possible modification can take place. Such modifications will be discussed in Section 1.15.

1.4 The I2C Bus Hardware Structure

The I2C bus physically consists of two active wires: a data line wire called serial data line (SDA) and a clock wire called serial clock line (SCL), and a ground connection, as shown in Figure 1.4. Sometimes, I2C is referred to as **Two-wire Interface** or the **TWI**, since it consists of two active wires. Both SDA and SCL are bidirectional. This means that in a particular device, these lines can be driven by the IC itself or from an external device. To achieve

this functionality, these signals use open-collector or open-drain outputs (depending on the technology). The bus interface is built around an input buffer and an open drain or open collector transistor (Figure 1.4). When the bus is IDLE, the bus lines are in the logic HIGH state. This is achieved by using an external pull-up resistor, which is a small current source. To put a signal on the bus, the chip drives its output transistor, thus pulling the bus to a LOW level. The two wires are acting accordingly as wired-AND. This concept achieves a “built-in” bus mastering technique. If the bus is “occupied” by a chip that is sending a 0, then all other chips lose their right to access the bus. This will be used for “bus arbitration” (see Section 1.19 for Bus Arbitration).

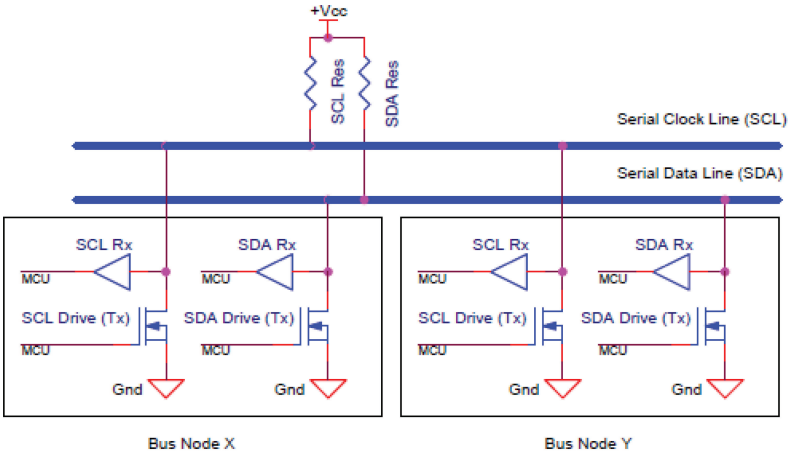
1.4.1 Electrical Consideration

Some electrical considerations must be taken by the designer of an I2C system. The two main considerations are as follows:

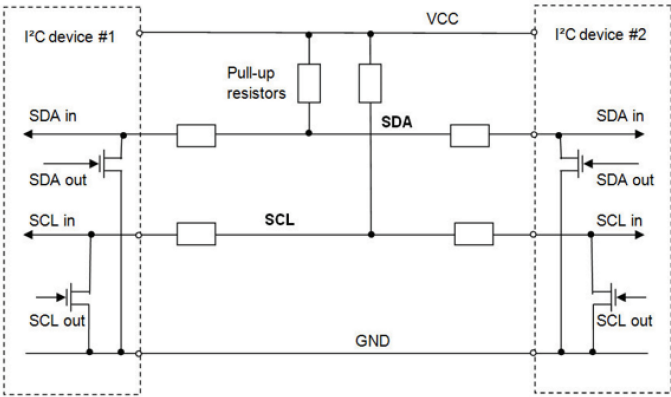
- The electrical considerations when connecting different devices to I2C bus
- The electrical considerations that limits the length of the bus

1.4.1.1 Types of devices that can be connected to I2C bus

The design of the I2C bus places no restrictions on the type of devices that may be connected to the bus. Figure 1.5 represents the hardware needed to connect two devices of the same logic family (e.g., both are using TTL or CMOS) using I2C bus. The two devices use the same supply voltage. The SCL is used to strobe data transmitted on SDA from or to the master that currently has control over the bus. Because the devices are wired-ANDed, this means that the bus is free if SDA and SCL are high and also that the device output is ANDed with the signal on the bus. This, as mentioned before, represents the basics of achieving the bus arbitration when there is more than one master. It is possible, in general to configure a system with a mixture of devices from different logic families, that is, a mixture of various TTL and CMOS logic families. In this case, each device has its own different supply voltage. To accommodate bidirectional communication with devices operating on different voltages, for example, 5 V and 3.5 V, it is necessary to incorporate into the SCL and SDA lines a form of buffering circuit as shown in Figure 1.6. The two devices, operating at different voltage levels, are now separated from one another.



(a)



(b)

Figure 1.5 I2C bus and interface hardware: Case of connecting two devices with the same supply voltage (www.freescale.com).

1.4.1.2 Electrical considerations that limits I2C bus length

There is no maximum length specified in the I2C specification. The limit of the length is a function of several factors including capacitance (usual maximum limit is 400 pF), the minimum value for R_p , propagation delays along the cables, the type of cable used and the integrity of the logic signals in the presence of noise. There are bus rise and fall times that must be met with limits set by the static and dynamic sink capability of standard I2C devices

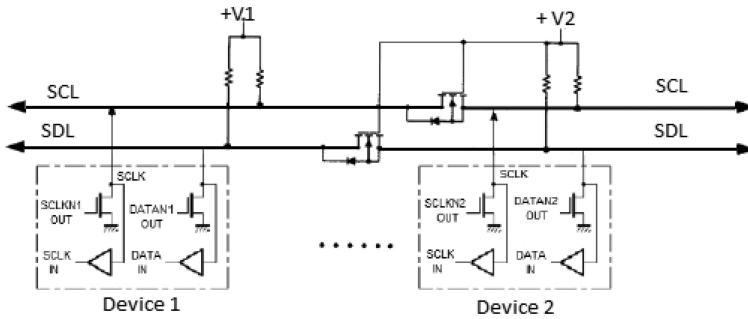


Figure 1.6 Connecting two devices with different supply voltage.

(3 mA sink current). Here we briefly discuss the main factors that affect the length of I2C bus which are as follows:

- Bus capacitance limitation
- Clock rise time limitation
- Noise pickup limitation

i. Limitation due to bus capacitance

The open-collector technique in use with I2C has a drawback that limits the length of the bus. Long lines present a capacitive load for the output drivers. Since the pull-up is passive, the RC constant of the bus will affect the wave shape of the signals propagating on the bus, which in turn limits the speed that the bus can handle; the higher the RC constant of the bus, the slower the speed of the bus. This results because of the effect of the RC constant on the slew rate of the edge of the signals. At a certain point, and because of the slew rate, the ICs will not be able to distinguish clearly between logic 1 and logic 0. Increasing the length of the line has another effect: it increases the possibility of getting reflections at high speed. The reflected waves can corrupt the data on the bus to the extent that it becomes unreadable.

It is important to note here that the bus capacitance includes not only the capacitance between each signal line (clock and data) to return but also the input capacitance of each device attached to the bus. This input capacitance can usually be found in a device's data sheet. For common ICs made with a CMOS process, the input capacitance is typically on the order of 10 pF. I2C specifications allows up to a maximum of 400 pF for the bus capacitance, the matter that put limitation on the maximum number to be connected to the bus.

To avoid such problems, a number of strict electrical specifications are to be followed when using I2C bus. According to the specification, the length

of the bus is limited by the total bus capacitance: the total capacitance of the bus remains under 400 pF.

ii. Clock rise time limitation

The RC constant of the bus will affect the wave shape of the signals propagating on the bus, which in turn limits the speed that the bus can handle; the higher the RC constant of the bus, the slower the speed of the bus. This results because of the effect of the RC constant on the slew rate of the edge of the signals. At a certain point, and because of the slew rate, the ICs will not be able to distinguish clearly between logic 1 and logic 0. Increasing the length of the line has another effect: it increases the possibility of getting reflections at high speed. The reflected waves can corrupt the data on the bus to the extent that they become unreadable.

For standard mode, the maximum rise time of the clock and data signals is 1000 ns. Note that the rise time is specified as the time the signal takes to rise from 30% to 70% of the supply voltage.

There are some ways to improve the rise time by using, for example, a switched pull-up circuit. In Section 2.4 of Chapter 2, some discussions concerning improving the rise time is given.

iii. Noise pickup limitation

Another issue is that a lot of devices generally means a long bus (more than about 20–25 cm), and a long bus often has problems with noise pickup. A common problem on noisy I2C buses is that a spike on the data line when the clock is high can cause the devices on the bus to see an improper START or STOP condition. This confuses the devices on the bus and can cause the bus to hang.

The original I2C specification allows a data transfer rates of up to 100 kbps and 7-bit addressing. Seven-bit addressing allows a total of 128 devices to communicate over a shared 1 bus. With increased data transfer rate requirements, the I2C specification has been recently enhanced (Version 1.0–1992 and Version 2.0–1998) to include fast mode, 3.4 Mbps, with 10-bit addressing. At present, I2C range includes more than 150 CMOS and bipolar I2C bus compatible types for performing communication functions between intelligent control devices (e.g., microcontrollers), general-purpose circuits (e.g., LCD drivers, thermal sensors, remote I/O ports, RAM, flash, and EPROM memories), and application-oriented circuits (e.g., digital tuning and signal processing circuits for radio and video systems). Other common devices capable of interfacing to an I2C bus include real-time clocks and watchdog timers.

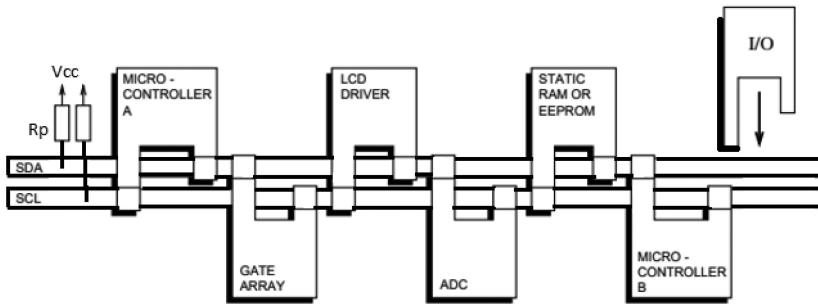


Figure 1.7 Basic configuration of I2C.

1.5 The Physical Bus–I2C Bus Interface

Figure 1.7 shows a typical I2C bus for an embedded system, where many different peripherals, acting as multiple slave devices, are sharing the bus. The microcontroller represents the I2C master, and controls the IO expanders, various sensors, EEPROM, ADCs/DACs, and much more. All of which are controlled with only 2 pins from the master. All the devices connected to the bus are classified as either being *Master* or *Slave* (just like SPI). The number of ICs that can be connected to the same bus is limited only by a maximum bus capacitance. More capacitance may be allowed under some conditions.

The possibility that many different peripherals may share one bus is one of the largest benefits that the I2C bus can give when compared to other interfaces. I2C bus is bidirectional data transfer with data rate depends on the mode of operation (standard mode, Fast mode, Fast-mode Plus).

Each device connected to the bus is software addressable by a unique address and simple master/slave relationships exist at all times; masters can operate as **master-transmitters** or as **master-receivers**.

Figure 1.8 highlights the master–slave and receiver–transmitter relationships found on the I2C bus. Note that these relationships are not permanent, but only depend on the direction of data transfer at that time. The transfer of data would proceed as follows:

1. Suppose microcontroller A wants to send information to microcontroller B:
 - Microcontroller A (master) addresses microcontroller B (slave).
 - Microcontroller A (**master-transmitter**) sends data to microcontroller B (slave-receiver)—microcontroller A terminates the transfer.

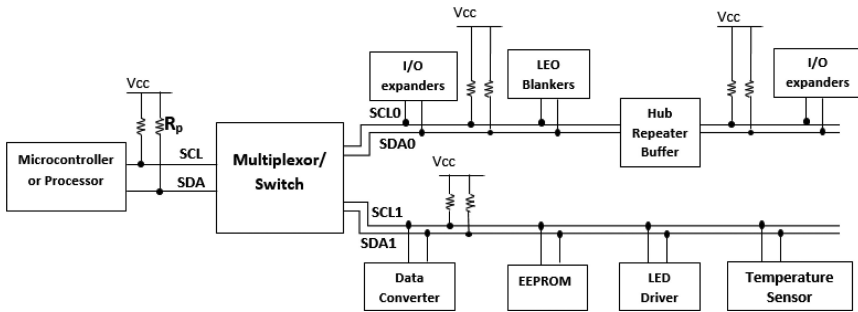


Figure 1.8 Application example showing I2C communication between the different ICs on a system and with pull-up resistors on I2C bus.

2. If microcontroller A wants to receive information from microcontroller B:
 - Microcontroller A (master) addresses microcontroller B (slave).
 - Microcontroller A (**master-receiver**) receives data from microcontroller B (slave-transmitter).
 - Microcontroller A terminates the transfer. Even in this case, the master (microcontroller A) generates the timing and terminates the transfer.

The possibility of connecting more than one microcontroller to the I2C bus means that more than one master could try to initiate a data transfer at the same time. To avoid the chaos that might ensue from such an event, I2C includes collision detection and arbitration to prevent data corruption if two or more masters simultaneously initiate data transfer.

Arbitration procedure relies on the wired-AND connection of all I2C interfaces to the I2C bus. If two or more masters try to put information onto the bus, the first to produce a “one” when the other produces a “zero” loses the arbitration. The clock signals during arbitration are a synchronized combination of the clocks generated by the masters using the wired-AND connection to the SCL line (for more detailed information concerning arbitration, see Section 1.19). Generation of clock signals on the I2C bus is always the responsibility of master devices; each master generates its own clock signals when transferring data on the bus. Bus clock signals from a master can only be altered when they are stretched by a slow slave device holding down the clock line or by another master device when arbitration occurs.

1.6 SDA and SCL Signals

Typically, both SDA and SCL are each connected to a 3.3 or 5 V power line through a single “pull-up” resistor, one on each line. This is necessary, because I2C devices have open-drain (or open-collector) lines to perform wired-AND function: they can force the voltage on the line to 0 V or “low,” but can’t raise it to 3.3 V or “high.” The pull-up resistor is used to ensure no component-harming high currents are able to flow when two devices try and signal simultaneously. High and low are the electrical representations of the 1s and 0s that are the fundamental components of digital information. Adding these two resistors—and the bus needs only two, no matter how many devices are connected to it—ensures the voltage rises back to 3.3 V without a short circuit.

In most cases, the designer will be expected to add these pull-up resistors, but some devices, typically those that operate at 3.3 V, include them in order to be compatible with devices supplying 5 V. Remembering that only one pair of pull-up resistors are needed per bus, it may be necessary to remove pull-up resistors attached to other devices on the bus.

The value of the pull-up resistor has direct effect on the rise time of the signals. Some companies like Texas Instrument introduced TCA980x Family to replace the resistance and to improve the rise time (see Chapter 2, Sections 2.2 and 2.3 for detail).

Data on the I2C bus can be transferred at rates of up to 100 kbps in the Standard mode, up to 400 kbps in the Fast mode, up to 1 Mbps in Fast mode Plus, or up to 3.4 Mbps in the High-speed mode. The bus capacitance limits the number of interfaces connected to the bus. For a single master application, the master’s SCL output can be a push–pull driver design if there are no devices on the bus which would stretch the clock.

1.7 Masters and Slaves

The I2C bus, as in case of ISP, separates devices into “masters” and “slaves.” A master is the device which initiates a data transfer on the bus and generates the clock signals to permit that transfer. All the other devices synchronize their timings to the master and are thus considered slaves. The master and its slaves can all transmit and receive data, but only the master can tell the slave when to transmit data back.

Both lines are connected to a positive supply via a *pull-up resistor* and remain HIGH when the bus is not busy. Each device is recognized by a unique

address—whether it is a microcomputer, LCD driver, memory or keyboard interface—and can operate as either a transmitter or receiver, depending on the function of the device. A device generating a message or data is a transmitter, and a device receiving the message or data is a receiver. Obviously, a passive function like an LCD driver could only be a receiver, while a microcontroller or a memory can both transmit and receive data. When a data transfer takes place on the bus, a device can either be a master or a slave. The device which initiates the transfer, and generates the clock signals for this transfer, is the master. At that time any device addressed is considered a slave. It is important to note that a master could either be a transmitter or a receiver; a master microcontroller may send data to a RAM acting as a transmitter and then interrogate the RAM for its contents acting as a receiver in both cases performing as the master initiating the transfer. In the same manner, a slave could be both a receiver and a transmitter. The I2C is a multi-master bus. It is possible to have, in one system, more than one device capable of initiating transfers and controlling the bus. A microcontroller may act as a master for one transfer, and then be the slave for another transfer, initiated by another processor on the network. The master/slave relationships on the bus are not permanent and may change on each transfer.

Just like SPI, it is possible to have multiple *Masters* connected to the same I2C bus. As more than one master may be connected to the bus, it is possible that two devices will try to initiate a transfer at the same time. Obviously, in order to eliminate bus collisions and communications chaos, an arbitration procedure is necessary. The I2C design has an inherent arbitration and clock synchronization procedure relying on the wired-AND connection of the devices on the bus. In a typical multi-master system, a microcontroller program should allow it to gracefully switch between master and slave modes and preserve data integrity upon loss of arbitration. Figure 1.8 is an application example of communication between different ICs.

1.7.1 Buffering and Multiplexing

When there are many I2C devices in a system, there can be a need to include bus buffers or multiplexers (see Figure 1.8) to split large bus segments into smaller ones. This can be necessary to keep the capacitance of a bus segment below the allowable value or to allow multiple devices with the same address to be separated by a multiplexer. Many types of multiplexers and buffers exist and all must take into account the fact that I2C lines are specified to be bidirectional. Multiplexers can be implemented with analog switches, which

can tie one segment to another. Analog switches maintain the bidirectional nature of the lines but do not isolate the capacitance of one segment from another or provide buffering capability.

Buffers can be used to isolate capacitance on one segment from another and/or allow I2C to be sent over longer cables or traces. Buffers for bidirectional lines such as I2C must use one of several schemes for preventing latch-up. I2C is open-drain, so buffers must drive a low on one side when they see a low on the other. One method for preventing latch-up is for a buffer to have carefully selected input and output levels such that the output level of its driver is higher than its input threshold, preventing it from triggering itself. For example, a buffer may have an input threshold of 0.4 V for detecting a low, but an output low level of 0.5 V. This method requires that all other devices on the bus have thresholds which are compatible and often means that multiple buffers implementing this scheme cannot be put in series with one another.

Alternatively, other types of buffers exist that implement current amplifiers or keep track of the state (i.e., which side drove the bus low) to prevent latch-up. The state method typically means that an unintended pulse is created during a hand-off when one side is driving the bus low, then the other drives it low, then the first side releases (this is common during an I2C acknowledgment). Buffering and multiplexing will be considered in Chapter 2.

1.8 I2C Data Validity

For the data to be valid on the SDA line, it must not change while the SCL is high. The data on the SDA line should change only and only when the SCL line goes low: Data on the SDA line must remain stable during the high phase of the clock period (Figure 1.9). If this standard is not followed, the data transfer becomes flawed, in which case it becomes a start/stop sequence.

1.9 Voltage Levels and Resistor Values

I2C supports a wide range of voltage levels, and hence, the designer can provide +5 V or +3.3 V as V_{cc} easily. The levels of logical “0” (LOW) and “1” (HIGH) are also not fixed and depend on the associated V_{cc} .

This gives the designer a wide range of choices for the values of the pull-up resistors (R_p). Anything within the range of 1 k to 47 k should be fine; however, values lower than 10 k are usually preferred.

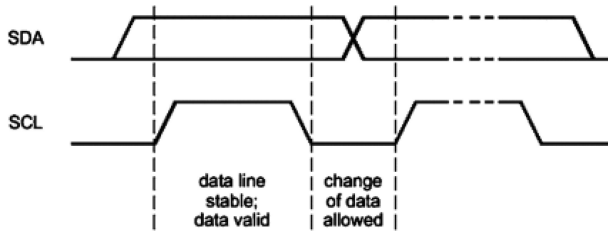


Figure 1.9 I2C data validity.

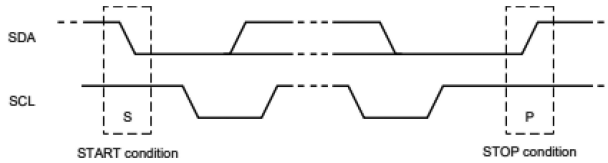


Figure 1.10 Start and stop conditions.

1.10 Start/Stop Sequence

All transactions on the bus begin with a START condition (S) and end with a STOP condition (P). In order for the *Master* to start talking to the *Slave(s)*, it must notify the *Slave(s)* about it. This is done using a special start sequence. While discussing I2C data validity, it was stated that the SDA should not change while the SCL is high. This rule doesn't hold for the start/stop sequence, which makes the start/stop sequence a special sequence.

When the SCL is high and SDA goes from high to low (as shown in the Figure 1.10), it marks the beginning of the transaction of *Master* with the *Slave(s)*.

And when the SDA goes from low to high while the SCL is still high (as shown in Figure 1.10), it marks the end of the transaction of that *Master* with the *Slave(s)*.

Detection of START and STOP conditions by devices connected to the bus is easy if they incorporate the necessary interfacing hardware. However, microcontrollers with no such interface have to sample the SDA line at least twice per clock period to sense the transition. START and STOP conditions are always generated by the master.

In-between the start and stop sequences, the bus is busy and no other *Master(s)* (if any) should try to initiate a transfer. The bus is considered to be free again a certain time after the STOP condition. This bus free situation is specified in I2C specifications. The bus stays busy if a repeated START

(Sr) is generated instead of a STOP condition. In this respect, the START (S) and repeated START (Sr) conditions are functionally identical. For the remainder of this document, therefore, the S symbol is used as a generic term to represent both the START and repeated START conditions, unless Sr is particularly relevant.

1.11 Repeated START Condition

A repeated START condition is similar to a START condition and is used in place of a back-to-back STOP then START condition. It looks identical to a START condition, but differs from a START condition because it happens before a STOP condition (when the bus is not idle). This is useful for when the master wishes to start a new communication, but does not wish to let the bus go idle with the STOP condition, which has the chance of the master losing control of the bus to another master (in multi-master environments).

1.12 Addressing Structure

1.12.1 7-bit Addressing

In order for one I2C device to communicate with another on a one-to-one basis, both devices need to be uniquely identifiable. This identity is the device's I2C address. The address comes directly after the START condition. I2C addresses are usually 7-bit numbers (Figure 1.11), so a bus can comprise up to 127 devices in all. As will be discussed, 10-bit address is also used.

When the *Master* calls out to the *Slave(s)*, it still needs to send out 8 bits of data. In this case, the *Master* appends an extra Read/Write (R/W') bit to the 7 bits of address (note that W' means Write complemented). The extra bit is used to indicate whether the signal is being sent by the master to the slave—a “write”—or in the other direction—a “read.” The R/W' bit is added to the LSB of the address byte. The 7-bit address is placed in bits one through



Figure 1.11 7-bit address.

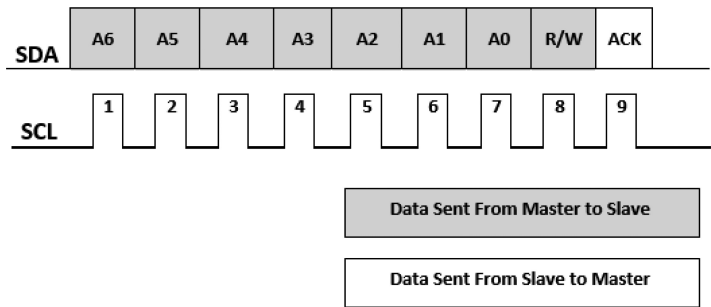


Figure 1.12 I2C device addressing.

Note: The legend shown at the bottom is only for SDA. SCL is always generated by the Master (whether transmitter or receiver).

Field:	S	I2C address field							R/W'	A	I2C message sequences...	P
Type	Start	Byte 1								ACK	Byte X etc...	Stop
Bit position in byte X		7	6	5	4	3	2	1	0			
7-bit address pos		7	6	5	4	3	2	1				
Note		MSB				LSB						

Figure 1.13 7-bit addressing structure.

seven of the address byte, and the data sent look as in Figure 1.12. The 7-bit addressing structure is shown in Figure 1.13.

1.12.2 Acknowledge Scheme

I2C transfers 8 bits (1 byte) of data at a time. After the transfer of each byte is complete, the receiver must acknowledge it. To acknowledge, the receiver sends an ACK bit back to the transmitter. This goes as follows:

- The transmitter (could be either *Master* or *Slave*) transmits 1 byte of data (MSB first) to the receiver during 8 clock pulses of SCL, after which it releases the SDA line, that is, the SDA line becomes HIGH for the ACK clock pulse.
- The receiver (could be either *Master* or *Slave*, it depends) is obliged to generate an acknowledgment after each byte sent by the transmitter by pulling the SDA line LOW for the ACK clock pulse (9th clock pulse) of SCL.

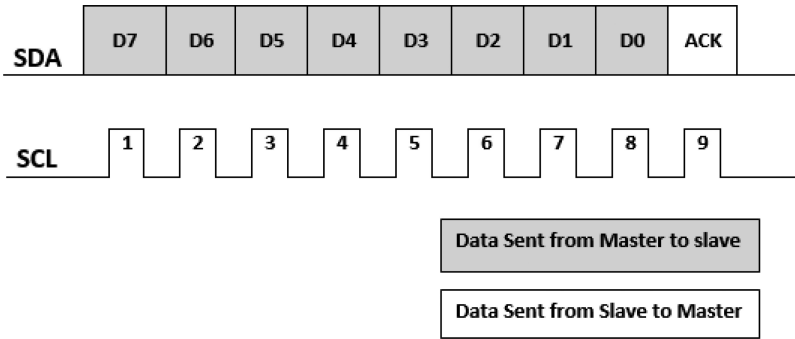


Figure 1.14 I2C acknowledgment scheme (Assumption: Master is the transmitter).

Note: The legend shown at the bottom is only for SDA. SCL is always generated by the Master (whether transmitter or receiver).

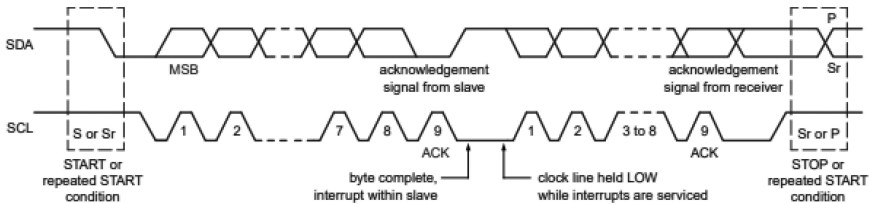


Figure 1.15 Data transfer on the I2C bus.

- So overall, there are 9 SCL clock pulses required to transmit a byte of data. This is shown in Figure 1.14 with the assumption that *Master* is the transmitter.

The number of bytes that can be transmitted per transfer is unrestricted. As mentioned before, each byte must be followed by an Acknowledge bit (see Figure 1.15).

Consider now the case of receiver fails to acknowledge receiving the data sent to it. There are two cases to that situation.

Case 1: Slave is at the receiver's end.

Under this case, it is possible to identify two possible cases:

- **CASE 1A:** The *Slave-receiver* does not acknowledge the *Slave* address (see “Address” latter). In that case, it simply leaves the SDA line HIGH. Now the *Master-transmitter* either generates a Stop sequence or attempts a repeated Start sequence.

- **CASE 1B:** The *Slave-receiver* acknowledges the *Slave* address, but after some time it is unable to receive any data and leaves the SDA line HIGH during the ACK pulse. Even in this case, the *Master-transmitter* does the same—either generates a Stop sequence or attempts a repeated Start sequence.

Case 2: Master is at the receiver's end.

In this case, the *Master* is the one generating ACK, as well as responsible for generating Start/Stop sequence. Now how does that work out, especially when the transaction ends?

In this case, in order to signal the *Slave-transmitter* the end of data, the *Master-receiver* does NOT generate any ACK on the last byte clocked out of the *Slave-transmitter*. In this case, the *Slave-transmitter* must let go of the SDA line to allow *Master* to generate a Stop or a repeated Start sequence.

1.12.3 I2C Addresses Standard: 10-bit Addressing

In order to prevent address clashes, due to the limited range of the 7-bit addresses, a new 10-bit address scheme has been introduced. This enhancement can be mixed with 7-bit addressing and increases the available address range about ten times. The 10-bit address uses two bytes (Figure 1.16). After the start condition, a leading “11110” introduces the 10-bit addressing scheme. The last two address bits of the first byte concatenated with the eight bits of the second byte of the whole 10-bit address. Devices which only use 7-bit addressing simply ignore messages with the leading “11110.” The following picture shows the first two bytes of a transfer with a 10-bit address.

A master transmitter addresses the slave with two address bytes as described above with the R/W' bit = “0” followed by data bytes from the master. The master receiver transfer is only possible with a Combined Transfer due to the fact that the second address byte can only be transmitted

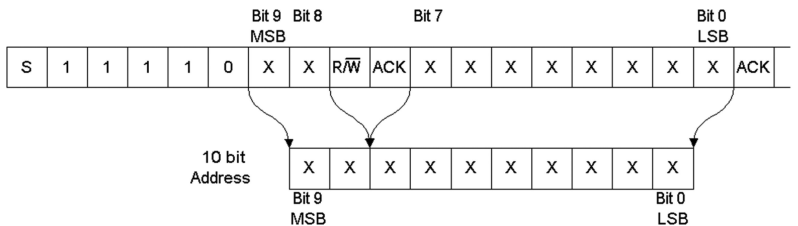


Figure 1.16 10-bit addressing.

Field:	S	10-bit mode indicator					Upper addr		R/W'	A	Lower address field								I2C message sequences	P
Type	Start	Byte 1								ACK	Byte 2								Byte X etc.	Stop
Bit position in byte X		7	6	5	4	3	2	1	0		7	6	5	4	3	2	1	0		
Bit value		1	1	1	1	0	X	X	X		X	X	X	X	X	X	X	X		
10-bit address pos							10	9												
Note		Indicates 10-bit mode					MSB		1 = Read 0 = Write								LSB	Rest of the read or write message goes here		

Figure 1.17 10-bit addressing structure.

if the R/W' bit of the first address byte is "0." Hence, the start of a master receiver transfer will be the same as a master transmitter transfer followed by a repeated start condition and the first byte of address byte with R/W' bit = "1" (switching to slave transmitter mode). Please refer to the following master receiver sequence:

- Start condition
- First address byte, R/W' bit = "0," ACK from the slave
- Second address byte, ACK from the slave
- Repeated start condition (no stop condition!)
- First address byte again, R/W' bit = "1," ACK from the slave, slave switches to transmit mode
- Slave transmits data bytes, ACK from master
- After the last data byte, the master sends a NACK
- Stop condition

Figure 1.17 shows the 10-bit addressing structure.

Practically, the 10-bit addressing has the following impact on the I2C protocol (refer to Figure 1.16):

- Two address words are used for device addressing instead of one.
- The first address word MSBs are conventionally coded as "11110" so any device on the bus is aware that the master sends a 10-bit device address.

1.12.3.1 I2C bus transactions in case of 10-bit address

All combinations of read/write formats previously described for 7-bit addressing (Section 1.13) are possible with 10-bit addressing. Two are



Figure 1.18 A master-transmitter addresses a slave-receiver with a 10-bit address.

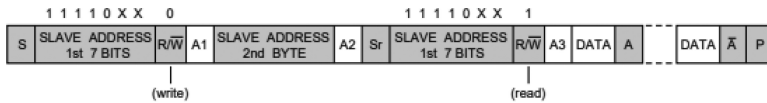


Figure 1.19 A master-receiver addresses a slave-transmitter with a 10-bit address.

detailed here:

- Master-transmitter transmits to slave-receiver with a 10-bit slave address.

The transfer direction is not changed (see Figure 1.18). When a 10-bit address follows a START condition, each slave compares the first seven bits of the first byte of the slave address (1111 0XX) with its own address and tests if the eighth bit (R/W direction bit) is 0. It is possible that more than one device finds a match and generate an acknowledge (A1). All slaves that found a match compare the eight bits of the second byte of the slave address (XXXX XXXX) with their own addresses, but only one slave finds a match and generates an acknowledge (A2). The matching slave remains addressed by the master until it receives a STOP condition (P) or a repeated START condition (Sr) followed by a different slave address.

- Master-receiver reads slave-transmitter with a 10-bit slave address.

The transfer direction is changed after the second R/W bit (Figure 1.19). Up to and including acknowledge bit A2, the procedure is the same as that described for a master-transmitter addressing a slave-receiver. After the repeated START condition (Sr), a matching slave remembers that it was addressed before. This slave then checks if the first seven bits of the first byte of the slave address following Sr are the same as they were after the START condition (S) and tests if the eighth (R/W) bit is 1. If there is a match, the slave considers that it has been addressed as a transmitter and generates acknowledge A3. The slave-transmitter remains addressed until it receives a STOP condition (P) or until it receives another repeated START condition (Sr) followed by a different slave address. After a repeated START condition (Sr), all the other slave devices will also compare the first seven bits of the first byte of the slave address (1111 0XX) with their own addresses and test

the eighth (R/W) bit. However, none of them will be addressed because $R/W = 1$ (for 10-bit devices), or the 1111 0XX slave address (for 7-bit devices) does not match.

Slave devices with 10-bit addressing react to a “general call” in the same way as slave devices with 7-bit addressing. Hardware masters can transmit their 10-bit address after a “general call.” In this case, the “general call” address byte is followed by two successive bytes containing the 10-bit address of the master-transmitter. The format is as shown in Figure 1.21 where the first DATA byte contains the eight least-significant bits of the master address.

The START byte 0000 0001 (01h) can precede the 10-bit addressing in the same way as for 7-bit addressing.

1.12.4 I2C Addresses Standard: Special Addresses and Exceptions in 7-bit Address Space

As mentioned before, the first byte of an I2C transfer contains the slave address and the data direction. The address is 7 bits long, followed by the direction bit (W/R bit). A 7-bit wide address allows, theoretically 128 I2C addresses—however, some addresses are reserved for special purposes. Only 112 addresses are available with the 7-bit address scheme.

As a matter of fact, I2C gives a loose standard for the address. It uses the most significant four bits to identify the type of the device and the next three bits are used to specify one of eight devices of this type (or further specify the device type). Also, some devices require certain patterns for the last three bits, while others (such as large serial EEPROMs) use these bits to specify an address inside the device. This shows the importance of mapping out the devices to be put on the bus and all their addresses. In the I2C address map, there are also what is called “*reserved addresses*.”

1.12.4.1 Reserved and none-reserved addresses in 7-bit address space

1.12.4.1.1 Reserved addresses in 7-bit address space

In case of 7-bit, two groups of eight addresses (0000 XXX and 1111 XXX) are reserved for the purposes shown in Table 1.1.

Assignment of addresses within a local system is up to the system architect who must take into account the devices being used on the bus and any future interaction with other conventional I2C buses. For example, a device with seven user-assignable address pins allows all 128 addresses to be assigned. If it is known that the reserved address is never going to be used for its intended purpose, a reserved address can be used for a slave address.

Table 1.1 Reserved addresses (X = don't care; 1 = high; 0 = low)

Slave Address	R/W'	Description
0000 000	0	General call address [1]
0000 000	1	START byte [2]
0000 001	X	CBUS address [3]
0000 010	X	Reserved for different bus format [4]
0000 011	X	Reserved for future purposes
0000 1XX	X	HS mode master code
1111 1XX	1	Device ID
1111 0XX	X	10-bit slave address

Notes:

- [1] The general call address is used for several functions including software reset (see next).
- [2] No device is allowed to acknowledge at the reception of the START byte.
- [3] The CBUS address has been reserved to enable the inter-mixing of CBUS compatible and I2C bus compatible devices in the same system. I2C bus compatible devices are not allowed to respond on reception of this address.
- [4] The address reserved for a different bus format is included to enable I2C and other protocols to be mixed. Only I2C bus compatible devices that can work with such formats and protocols are allowed to respond to this address.

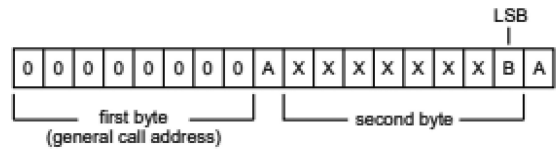


Figure 1.20 General call address format.

i. General call address

The general call address is for addressing every device connected to the I2C bus at the same time. However, if a device does not need any of the data supplied within the general call structure, it can ignore this address by not issuing an acknowledgment. If a device does require data from a general call address, it acknowledges this address and behaves as a slave-receiver. The master does not actually know how many devices are acknowledged if one or more devices respond. The second and following bytes are acknowledged by every slave-receiver capable of handling these data. A slave who cannot process one of these bytes must ignore it by not acknowledging. Again, if one or more slaves acknowledge, the not acknowledged will not be seen by the master. The meaning of the general call address is always specified in the second byte (see Figure 1.20).

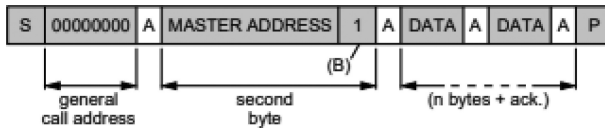


Figure 1.21 Data transfer from a hardware master-transmitter.

There are two cases to consider:

- When the least significant bit B is a “zero.”
- When the least significant bit B is a “one.”

When bit B is a “zero,” the second byte has the following definition:

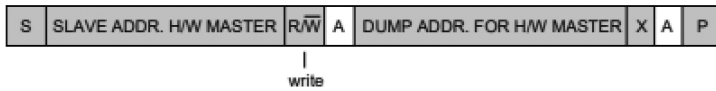
- 0000 0110 (06h): Reset and write programmable part of slave address by hardware. On receiving this 2-byte sequence, all devices designed to respond to the general call address reset and take in the programmable part of their address. Precautions must be taken to ensure that a device is not pulling down the SDA or SCL line after applying the supply voltage, since these low levels would block the bus.
- 0000 0100 (04h): Write programmable part of slave address by hardware. Behaves as above, but the device does not reset.
- 0000 0000 (00h): This code is not allowed to be used as the second byte.

Sequences of programming procedure are published in the appropriate device data sheets. The remaining codes have not been fixed and devices must ignore them.

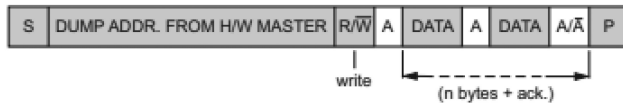
When bit B is a “one,” the 2-byte sequence is a “hardware general call.” This means that the sequence is transmitted by a hardware master device, such as a keyboard scanner, which can be programmed to transmit a desired slave address. Since a hardware master does not know in advance to which device the message has to be transferred, it can only generate this hardware general call and its own address—identifying itself to the system (see Figure 1.21).

The seven bits remaining in the second byte contain the address of the hardware master. This address is recognized by an intelligent device (e.g., a microcontroller) connected to the bus which then accepts the information from the hardware master. If the hardware master can also act as a slave, the slave address is identical to the master address.

In some systems, an alternative could be that the hardware master transmitter is set in the slave-receiver mode after the system reset. In this way, a system configuring master can tell the hardware master-transmitter (which is now in slave-receiver mode) to which address data must be sent (see



a. Configuring master sends dump address to hardware master



b. Hardware master dumps data to selected slave

Figure 1.22 Data transfer by a hardware-transmitter capable of dumping data directly to slave devices.

Figure 1.22). After this programming procedure, the hardware master remains in the master-transmitter mode.

ii. Software reset

Following a general call address (0000 0000), sending 0000 0110 (06h) as the second byte causes a software reset. This feature is optional and not all devices respond to this command. On receiving this 2-byte sequence, all devices designed to respond to the general call address reset and take in the programmable part of their address. Precautions must be taken to ensure that a device is not pulling down the SDA or SCL line after applying the supply voltage, since these low levels would block the bus.

iii. START byte

Microcontrollers can be connected to the I2C bus in two ways. A microcontroller with an on-chip hardware I2C bus interface can be programmed to be only interrupted by requests from the bus. When the device does not have such an interface, it must constantly monitor the bus via software. Obviously, the more times the microcontroller monitors, or polls the bus, the less time it can spend carrying out its intended function. There is therefore a speed difference between fast hardware devices and a relatively slow microcontroller which relies on software polling. In this case, data transfer can be preceded by a start procedure which is much longer than normal (see Figure 1.23). The start procedure consists of:

- A START condition (S)
- A START byte (0000 0001)

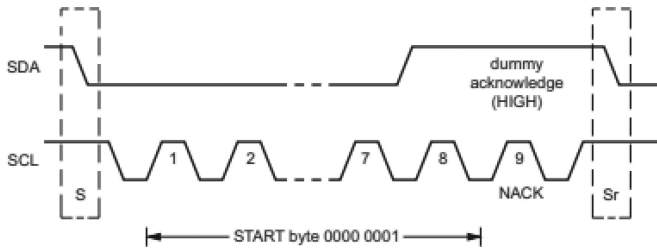


Figure 1.23 START byte procedure.

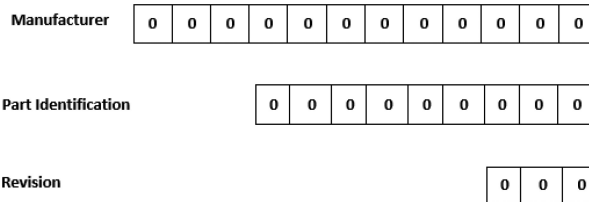


Figure 1.24 Device ID field.

- An acknowledge clock pulse (ACK)
- A repeated START condition (Sr).

After the START condition S has been transmitted by a master which requires bus access, the START byte (0000 0001) is transmitted. Another microcontroller can therefore sample the SDA line at a low sampling rate until one of the seven zeros in the START byte is detected. After the detection of this LOW level on the SDA line, the microcontroller can switch to a higher sampling rate to find the repeated START condition Sr which is then used for synchronization. A hardware receiver resets upon receipt of the repeated START condition Sr and therefore ignores the START byte. An acknowledge-related clock pulse is generated after the START byte. This is present only to conform with the byte handling format used on the bus. No device is allowed to acknowledge the START byte.

iv. Device ID

The Device ID field (see Figure 1.24) is an optional 3-byte read-only (24 bits) word giving the following information:

- Twelve bits with the manufacturer name, unique per manufacturer (e.g., NXP)
- Nine bits with the part identification, assigned by manufacturer (e.g., PCA9698)

- Three bits with the die revision, assigned by manufacturer (e.g., RevX)

The Device ID is read-only, hard-wired in the device and can be accessed as follows:

1. START condition
2. The master sends the Reserved Device ID I2C bus address followed by the R/W bit set to “0” (write): “1111 1000.”
3. The master sends the I2C bus slave address of the slave device it must identify. The LSB is a “Don’t care” value. Only one device must acknowledge this byte (the one that has the I2C bus slave address).
4. The master sends a Re-START condition. Remark: A STOP condition followed by a START condition resets the slave state machine and the Device ID Read cannot be performed. Also, a STOP condition or a Re-START condition followed by an access to another slave device resets the slave state machine and the Device ID Read cannot be performed.
5. The master sends the Reserved Device ID I2C bus address followed by the R/W bit set to “1” (read): “1111 1001.”
6. The Device ID Read can be done, starting with the 12 manufacturer bits (first byte + four MSBs of the second byte), followed by the nine part identification bits (four LSBs of the second byte + five MSBs of the third byte), and then the three die revision bits (three LSBs of the third byte).
7. The master ends the reading sequence by NACKing the last byte, thus resetting the slave device state machine and allowing the master to send the STOP condition. Remark: The reading of the Device ID can be stopped anytime by sending a NACK.

If the master continues to ACK the bytes after the third byte, the slave rolls back to the first byte and keeps sending the Device ID sequence until a NACK has been detected.

1.12.4.2 Non-reserved addresses in 7-bit address space

Table 1.3 shows the non-reserved addresses in the 7-bit address space.

Although MSB 1111 is reserved for Device ID and 10-bit slave addressing, it is also used by VESA DDC display-dependent devices such as pointing devices.

1.13 I2C Bus Transaction

After discussing the I2C bus interface, it is important to explain how the data transfer actually takes place through that interface. I2C supports

Table 1.2 Assigned manufacturer IDs

Manufacturer Bits											Company
11	10	9	8	7	6	5	4	3	2	1	
0	0	0	0	0	0	0	0	0	0	0	NXP Semiconductors
0	0	0	0	0	0	0	0	0	0	1	NXP Semiconductors (reserved)
0	0	0	0	0	0	0	0	0	1	0	NXP Semiconductors (reserved)
0	0	0	0	0	0	0	0	0	1	1	NXP Semiconductors (reserved)
0	0	0	0	0	0	0	0	1	0	0	Ramtron International
0	0	0	0	0	0	0	0	1	0	1	Analog Devices
0	0	0	0	0	0	0	0	1	1	0	STMicroelectronics
0	0	0	0	0	0	0	0	1	1	1	ON Semiconductor
0	0	0	0	0	0	0	1	0	0	0	Sprintek Corporation
0	0	0	0	0	0	0	1	0	0	1	ESPROS Photonics AG
0	0	0	0	0	0	0	1	0	1	0	Fujitsu Semiconductor
0	0	0	0	0	0	0	1	0	1	1	Flir
0	0	0	0	0	0	0	1	1	0	0	O ₂ Micro
0	0	0	0	0	0	0	1	1	0	1	Atmel

Table 1.3 Non-reserved addresses in the 7-bit address space

MSB (4-bit)	Typical Usage
0001	Digital receivers, SMBus
0010	TV video line decoders, IPMB
0011	AV codecs
0100	Video encoders, GPIO expanders
0101	ACCESS bus, PMBus
0110	VESA DDC, PMBus
0111	Display controller
1000	TV signal processing, audio processing, SMBus
1001	AV switching, ADCs and DACs, IPMB, SMBus
1010	Storage memory, real-time clock
1011	AV processors
1100	PLLs and tuners, modulators and demodulators, SMBus
1101	AV processors and decoders, audio power amplifiers, SMBus
1110	AV color space converters

unidirectional as well as bidirectional data transfer. There are three possible cases of data transfer, normally called bus events:

- Unidirectional Data Transfer
 - *I2C Bus Events 1: Master-transmitter to Slave-receiver (Case 1)*
 - *I2C Bus Events 2: Slave-transmitter to Master-receiver (Case 2)*
- Bidirectional Data Transfer
 - *I2C Bus Events 3: Master to Slave and Slave to Master (Case 3)*

1.13.1 I2C Bus Events 1: Master (Transmitter) to Slave (Receiver) Data Transfer

This is the event when the master transmitting a byte to a slave device. In other words, it is the case when the master writes to the slave. The transaction in this case will be as in Figure 1.25.

To write to a slave, the write cycle is as follows:

- The master device initiates the transfer by a start condition. This acts as an “Attention” signal to all of the connected devices. All ICs on the bus will listen to the bus for incoming data.
- Once the start condition has sent, the master sends a byte to the slave. This first byte is used to identify the slave on the bus and to select the mode of operation. As shown in Figure 1.25, the 7 bits forming the address of the slave starting with the most significant bit (MSB) is sent at first followed by the eighth bit that defines the direction of the data. The eighth bit is labeled R/W in the figure with 1 for “read” and 0 for “write”. In this case, the eighth bit is set to zero because the Master is writing to the slave. Here, the bit value is placed on the SDA line by the master device, while the SCL line is low and maintained stable until after a clock pulse on SCL.

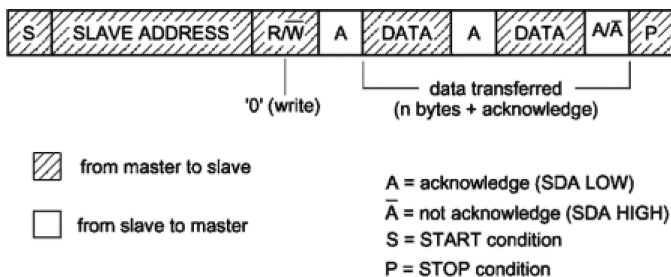


Figure 1.25 Master-to-slave data transfer.

- Having received the address, all the ICs will compare it with their own address. If there is no match, the ICs will ignore the rest of this transaction and wait for the next, that is, wait until the bus is released by the stop condition. On the other hand, if the address matches, the receiving device will respond by producing an acknowledge signal. The receiving device produces this signal by holding the SDA line low during the first ACK clock cycle.
- Once the *Slave* acknowledges the address, *Master* can now send data to the *Slave* byte-by-byte. The *Slave* has to send the ACK bit after every byte it receives. The master device transmits a byte of data starting with the most significant bit down to the least significant bit. The receiving device, in this case one of the slaves, acknowledges the reception of data by holding the SDA line low during the second ACK clock cycle. This means that for every 8 bits transferred, the device receiving the data sends back an acknowledge bit, so there are actually 9 SCL clock pulses to transfer each 8 bit byte of data. If the receiving device sends back a low ACK bit, then it has received the data and is ready to accept another byte.
- This goes on till *Slave* can no longer receive data and does NOT send the ACK bit.
- This indicates that the slave cannot accept any further data and the master should terminate the transfer by sending a stop sequence.

We see that the data transfer never changes its direction. Data always flow from *Master* to *Slave*, which makes the setup quite easy.

An example of this case would be like performing page write operations on an EEPROM chip.

1.13.2 I2C Bus Events 2: Slave (Transmitter) to Master (Receiver) Data Transfer

This is the Bus Event when the master reads a byte from a slave device. The transaction in this case is shown in Figure 1.26.

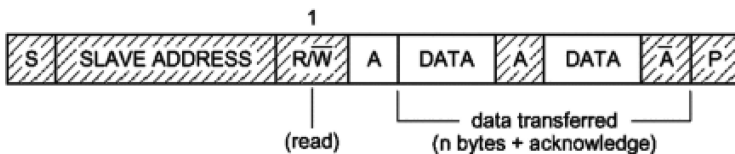
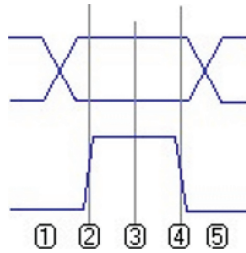


Figure 1.26 Slave-to-master data transfer.

**Figure 1.27** Read event.

To read from a slave device, the read cycle will be as follows:

- If performing a read operation, the master initiates the transfer by a start condition, sends the address of the device that is being read, sends a one (logic high on SDA line) requesting a read and waits to receive an acknowledgment.
- The *Slave* acknowledges the address, thus ready to send data now.
- The protocol format is the same as in transmitting a byte to a slave, except that now the master is not allowed to touch the SDA line. Prior to sending the 8 clock pulses needed to clock in a byte on the SCL line, the master releases the SDA line. The slave will now take control of this line. The line will then go high if it wants to transmit a “1,” or if the slave wants to send a “0,” the line will remain low.
- All the master has to do (see Figure 1.27) is to generate a rising edge on the SCL line (2), read the level on SDA (3) and generate a falling edge on the SCL line (4). The slave will not change the data during the time that SCL is high. (Otherwise a Start or Stop condition might inadvertently be generated.) During (1) and (5), the slave may change the state of the SDA line.
- In total, this sequence has to be performed 8 times to complete the data byte.

The meaning of all bytes being read depends on the slave. There is no such thing as a “universal status register.” The user needs to consult the data sheet of the slave being addressed to know the meaning of each bit in any byte transmitted.

- *Slave* keeps on sending data to the *Master*, and the *Master* keeps on sending ACK to the *Slave* after each byte until it can no longer accept any more data.

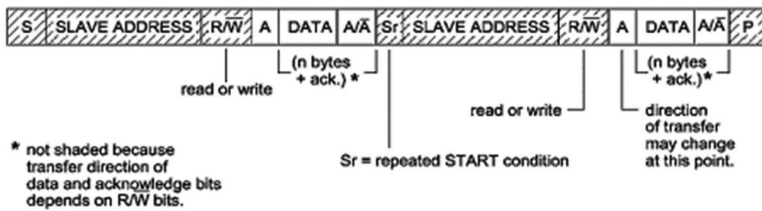


Figure 1.28 Bidirectional data transfer.

- When the *Master* feels like ending the transaction, it *does not* send the ACK, thus ending with the STOP sequence.

An example of this case could be an Analog to Digital Converter (ADC) sending data to the microcontroller continuously. The microcontroller accepts data as long as it wants to, after which it stops/finishes execution.

1.13.3 I2C Bus Events 3: Bidirectional Read and Write in Same Data Transfer

The transaction is as in Figure 1.28.

- The *Master* sends out the START sequence, followed by the 7-bit *Slave* address and the R/W' bit.
- The *Slave* acknowledges the address.
- Depending upon the value of the R/W' bit, read/write operations are performed (like the above two cases).
- Whatever the case it may be, it always ends with the receiver *not* sending the ACK.
- Until now, in the previous two cases, we have seen that the *Master* would close the connection. But in this case, the *Master* attempts a repeated START.
- And the entire process repeats again, until the *Master* decides to STOP.

As we can see, a change of direction of data transfer might happen depending upon the R/W' bits in the entire transaction.

An example of this case could be performing sequential read from an EEPROM chip. It is bidirectional because the CPU first *writes* the address from where it would like to start reading, followed by *reading* from the device. It is like, unless you tell the device from where you would like to start reading, how would it start sending you the data?

1.14 Clock Stretching

The above discussion concerning timing is valid as the *slave* is able to return to the *Master* the data needed. In some cases, the *Slave* is not ready yet to return the data to the *Master*. With devices like ADC or EEPROM, this is not an issue, but it is an issue with devices as microcontroller.

What if the *Slave* is a microcontroller, and the *Master* requests for a data which is not there in its cache? This would require the microcontroller to perform context switching, force it to search for it in the RAM, store it back in cache and then send it to the *Master*. This could (and definitely would) take a much longer time than the clock pulses of the SCL, and everything would just go wrong!

To solve such situation, I2C introduces what is called “*Clock Stretching*.” An addressed slave device may hold the clock line (SCL) low after receiving (or sending) a byte, indicating that it is not yet ready to process more data. The master communicating with the slave will try to raise the clock to transfer the next bit but will verify that the clock line will remain low. The master will then have to wait for the slave to release the line (SCL goes high) so that the clock signal can be transmitted. Normally, the master waits an additional minimum time of 4 μ s for standard 100 kbps I2C before pulling the clock low again. If a master wants to slow down the rate of data transfer, it just delays the next clock edge.

Although the master may also hold the SCL line low for as long as it desires (this is not allowed in newest Rev. 6 of the protocol—Section 3.1.1), the term “clock stretching” is normally used only when slaves do it. Although in theory any clock pulse may be stretched, generally it is the intervals before or after the acknowledgment bit which are used. For example, if the slave is a microcontroller, its I2C interface could stretch the clock after each byte, until the software decides whether to send a positive acknowledgment or a NACK.

Clock stretching is the only instance in the entire I2C protocol where the *Slave* drives the clock line SCL. In many processors and microcontrollers, the low-level hardware does this for us, so that we don’t have to worry about it while writing the code.

Many slaves do not need to clock stretch and thus treat SCL as strictly an input with no circuitry to drive it. Some masters, such as those found inside custom ASICs may not support clock stretching; often, these devices will be labeled as a “two-wire interface” and not I2C.

To ensure a minimal bus throughput, SMBus places limits on how far clocks may be stretched. Hosts and slaves adhering to those limits cannot

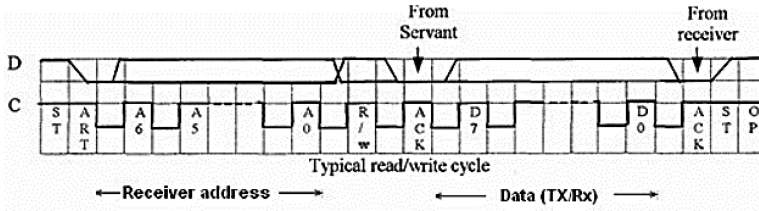


Figure 1.29 Read/write cycle.

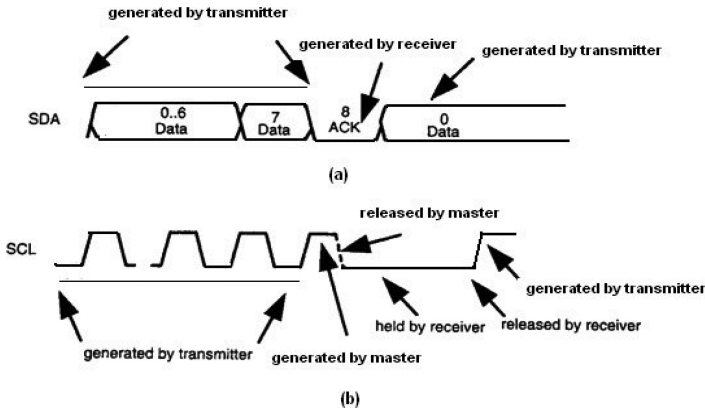


Figure 1.30 Acknowledge extension (a) without and (b) with extension.

block access to the bus for more than a short time, which is not a guarantee made by pure I2C systems.

1.15 Possible Modifications on the Timing Diagram

Figure 1.29 depicts a timing diagram of a typical read/write cycle. Sometimes, this timing diagram may change. Some of the possible changes are (see Figure 1.30):

- **Acknowledge Extension:** Under normal circumstances, following the ACK bit time, the master will release the SCL line so that transmission may continue with the next byte. If, however, the receiver (which is usually a “slave device” or “master”) is temporarily unable to proceed, it will hold the SCL line LOW, thereby extending the ACK interval. In other words, it is allowed for the acknowledge bit to float high. When the receiver is ready to proceed again, it will release the SCL line and transmission continues. The timing diagram in Figure 1.30 illustrates the ACK interval extension (see “Clock Stretching”).

- **Multi-register Slaves:** While explaining the write/read cycle, we assumed that the slave is simple and has only one register. In such a case, just sending the address of the slave is enough. In many cases, the slave has many registers, and each has its own address within the slave device. If the master wants to write at specific register within the slave, it must send this address. In such cases, having addressed the slave device, the master must now send out the internal location or register number inside the slave that it wishes to write to or read from. This number is obviously dependent on what the slave actually is and how many internal registers it has. In such a case, the master can continue to send data bytes to the slave, and these will normally be placed in the following registers because the slave will automatically increment the internal register address after each byte. When the master has finished writing all data to the slave, it sends a stop sequence which completes the transaction.
- **Repeated Start:** During an I2C transfer, there is often the need to first send a command and then read back an answer right away. This has to be done without the risk of another (multi-master) device interrupting this atomic operation. The I2C protocol defines a so-called repeated start condition.

After having sent the address byte (address and read/write bit), the master may send any number of bytes followed by a stop condition. If, however, it wishes to establish a connection with a different slave, rather than issue the Stop, the master will issue another Start, using the address of the new device and then sends more data. This is defined recursively allowing any number of start conditions to be sent. The purpose of this is, in general, to allow combined write/read operations to one or more devices without releasing the bus and thus with the guarantee that the operation is not interrupted.

- Also we have to mention here that, in some devices, a start bit has to be resent to reset the receiving device for the next command, for example, in a serial EEPROM read, the first command sends the address to read from, the second reads the data at that address. Multiple address is shown in Figure 1.31.

1.16 Bus Clear

In the unlikely event where the clock (SCL) is stuck LOW, the preferential procedure is to reset the bus using the HW reset signal if I2C devices in

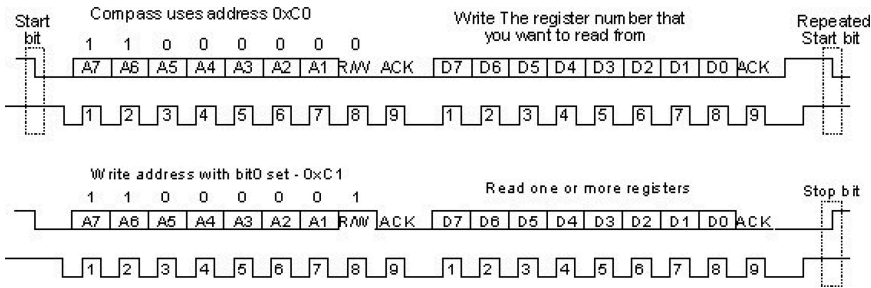


Figure 1.31 Multiple address.

use have HW reset inputs. If the I2C devices do not have HW reset inputs, cycle power of the devices activate the mandatory internal Power-On Reset (POR) circuit. In such cases, data line (SDA) is stuck LOW, the master should send nine clock pulses. The device that held the bus LOW should release it sometime within those nine clocks. If not, then use the HW reset or cycle power to clear the bus.

1.17 Applicability of I2C Bus Features

The above sections discussed the main features of the I2C bus protocol. Some of the features are mandatory and some are optional based on if the bus is single master or multi-master. Table 1.4 summarizes the use of mandatory and optional portions of the I2C bus specification and which system configurations use them.

1.18 I2C Modes: Bus Speeds

The first I2C introduced by Philips is considered as the standard mode. It refers to the initial transfer speed mode of the I2C specification which allows up to 100 kbps. There are applications where the I2C transfer speed is a limiting factor. To allow for higher transmission rates while retaining a certain amount of compatibility, Philips in 1998 has introduced the high-speed (HS) I2C standard and continued to introduce new standards. Currently, there are five operating speed categories (modes):

1. Standard mode (Low-speed mode) with a bit rate up to 100 kbps
2. Fast mode (FM) with a bit rate up to 400 kbps
3. Fast-mode plus (FM+) with a bit rate up to 1 Mbps

Table 1.4 Applicability of I2C bus protocol features

Feature	Configuration		
	Single Master	Multi-master	Slave*
START condition	M	M	M
STOP condition	M	M	M
Acknowledge	M	M	M
Synchronization	n/a	M	n/a
Arbitration	n/a	M	n/a
Clock stretching	O **	O **	O
7-bit slave address	M	M	M
10-bit slave address	O	O	O
General call address	O	O	O
Software reset	O	O	O
START byte	n/a	O***	n/a
Device ID	n/a	n/a	O

M = mandatory; O = optional; n/a = not applicable.

Notes: [*] Also refers to a master acting as a slave. [**] Clock stretching is a feature of some slaves. If no slaves in a system can stretch the clock (hold SCL Low), the master need not be designed to handle this procedure. [***] “Bit banging” (software emulation) multi-master systems should consider a START byte.

4. High-speed mode (HS-mode) with a bit rate up to 3.4 Mbps
5. Ultra-fast mode (UFm) with a bit rate up to 5 Mbps

The first four are bidirectional and compatible: any device may be operated at a lower bus speed. Ultra-fast mode devices are not compatible with the first four modes since the bus is unidirectional.

1.18.1 Low-speed Mode or Standard Mode

The very first specification of I2C dates back to the year 1982. It only covered Standard mode: up to 100 kbps and 7-bit addressing.

1.18.2 Enhanced I2C (Fast Mode)

In the fast mode, the physical bus parameters are not altered. The protocol, bus levels, capacitive load, etc., remain unchanged. However, the data rate has been increased to 400 kbps and a constraint has been set on the level of noise that can be present in the system. To accomplish this task, a number of changes have been made to the I2C bus timing.

The fast-mode I2C bus specification has the following additional features compared with the standard mode:

- The maximum bit rate is increased to 400 kbps.
- Timing of the serial data (SDA) and serial clock (SCL) signals has been adapted. There is no need for compatibility with other bus systems such as CBUS because they cannot operate at the increased bit rate.
- The inputs of fast-mode devices incorporate spike suppression and a Schmitt trigger at the SDA and SCL inputs.
- The output buffers of fast-mode devices incorporate slope control of the falling edges of the SDA and SCL signals.
- If the power supply to a fast-mode device is switched off, the SDA and SCL I/O pins must be floating so that they do not obstruct the bus lines.

1.18.3 Fast-mode plus (FM+)

FM+ devices introduced by Philips Semiconductors (now: NXP) in April 2006 can transfer information at rates up to 1 Mbps. The *FM+* devices are fully downward compatible with Fast- or Standard-mode devices for bidirectional communication in a mixed-speed bus system. The same serial bus protocol and data format is maintained as with the Fast- or Standard-mode system.

The I2C fast mode was originally specified to run at 400 kHz. Ignoring the 400 kilobit limit and running the same bus with the same logic at higher speeds faces a couple of factors which limit the maximum speed on the bus. First, there's the capacitance which introduces a rise time for the signals. As a "counter measure" the current on the bus can be increased by lowering the resistor values for the termination pull-ups.

Second, the bus has some timing tolerances (tolerant of the 1 μ s rise time), and if the speed is increased, these tolerances need to be more restrictive.

In applications where only Fast-mode Plus parts are present, the high drive strength and tolerance for slow rise and fall times allow the use of larger bus capacitance as long as set-up, minimum LOW time and minimum HIGH time for Fast-mode Plus are all satisfied, and the fall time and rise time do not exceed the 300 ns t_f and 1 μ s t_r specifications of Standard mode. Bus speed can be traded against load capacitance to increase the maximum capacitance by about a factor of ten.

Unlike the high speed, there is no additional logic to implement while using *FM+*. Therefore, fast-mode plus devices are downward compatible with

standard and fast-mode devices. In fact, many I2C slaves on the market like RAMs and EEPROMs will already tolerate higher bus frequencies.

The specification for FM+ is now part of the current I²C-bus specification and user manual.

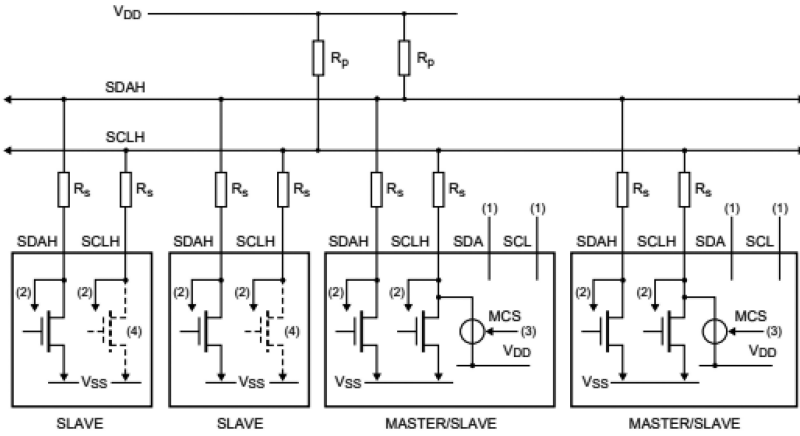
1.18.4 High-speed Mode (HS mode)

High-speed mode (HS mode) devices offer a quantum leap in I2C bus transfer speeds. HS-mode devices can transfer information at bit rates of up to 3.4 Mbps, yet they remain fully downward compatible with Fast- or Standard mode (F/S-mode) devices for bidirectional communication in a mixed-speed bus system. With the exception that arbitration and clock synchronization is not performed during the HS-mode transfer, the same serial bus protocol and data format is maintained as with the F/S-mode system. Depending on the application, new devices may have a Fast- or Hs-mode I2C bus interface, although HS-mode devices are preferred as they can be designed-in to a greater number of applications.

1.18.4.1 Electrical characteristics of HS mode

To achieve a bit transfer of up to 3.4 Mbps, HS devices have the following electrical characteristics:

- The high-speed variant of the I2C bus allows communication up to 3.4 Mbps.
- Both master and slave devices must be high-speed-enabled in order to benefit from this increase.
- High-speed IC devices are downward compatible allowing for mixed bus systems.
- In order to shorten signal rise time, HS-mode master devices have an open-drain output buffer for the SDAH signal and a combination of an open-drain pull-down and current-source pull-up circuit on the SCL output. It also has an open-drain output buffer for SDA signal. Only the current source of one master is enabled at any one time and only during HS mode.
- HS IC masters can actually source current to the bus which is referred to as boosting.
- This current source is enabled only during HS operation and just for one master.
- HS-mode master devices generate a serial clock signal with a HIGH to LOW ratio of 1 to 2. This relieves the timing requirements for set-up and hold times.



- (1) SDA and SCL are not used here but may be used for other functions.
- (2) To input filter.
- (3) Only the active master can enable its current-source pull-up circuit.
- (4) Dotted transistors are optional open-drain outputs which can stretch the serial clock signal SCLH.

Figure 1.32 I2C bus configuration with HS devices.

- HS-mode master devices can have a built-in bridge to separate lower speed devices from the bus during HS transfer. The main purpose of such bridge is to reduce the capacitive load on the bus and to avoid conflicts caused by low-speed devices.
- No arbitration or clock synchronization is performed during HS-mode transfer in multi-master systems, which speeds-up bit handling capabilities. The arbitration procedure always finishes after a preceding master code transmission in F/S-mode.

Figure 1.32 shows the physical I2C bus configuration in a system with only HS-mode devices. The pins, SDA and SCL, on the master devices are only used in mixed-speed bus systems and are not connected in an Hs-mode only system. In such cases, these pins can be used for other functions.

In Figure 1.32, an optional series resistor R_s is used. The optional series resistors R_s protect the I/O stages of the I2C bus devices from high-voltage spikes on the bus lines and minimize ringing and interference. Pull-up resistors R_p maintain the SDAH and SCLH lines at a HIGH level when the bus is free and ensure that the signals are pulled up from a LOW to a HIGH level within the required rise time. For higher capacitive bus-line loads (>100 pF), the resistor R_p can be replaced by external current source pull-ups to meet the rise time requirements. Unless preceded by an acknowledge bit, the rise time



Figure 1.33 Details of the first part of high speed transfer format.

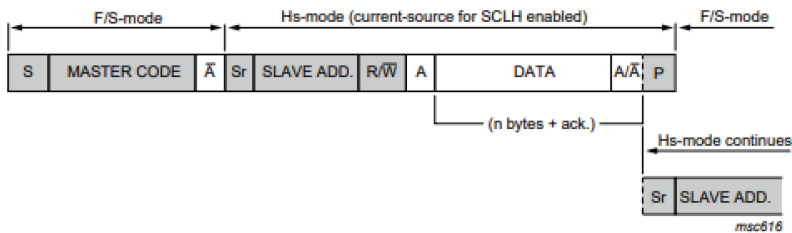


Figure 1.34 Data transfer format in HS-mode.

of the SCLH clock pulses in HS-mode transfers is shortened by the internal current-source pull-up circuit MCS of the active master.

1.18.4.2 Transmission format of high-speed mode

Serial data transfer format in HS-mode meets the Standard-mode I2Cbus specification. HS-mode can only commence after the following conditions (all of which are in F/S-mode):

1. START condition (S)
2. 8-bit master code (0000 1XXX)
3. Not-acknowledged bit (A)

Figure 1.33 shows the beginning of a high-speed transfer. This master code has two main functions:

- It allows arbitration and synchronization between competing masters at F/S-mode speeds, resulting in one winning master.
- It indicates the beginning of an HS-mode transfer.

Figures 1.34 and 1.35 show the transmission format in more detail. The format is as follows:

- Start condition is sent.
- After the start condition a so-called master code is transmitted “00001XXX” (see Table 1.1), followed by a mandatory not-acknowledge bit. The master code is sent in Fast- or Standard-mode (this is with at most 400 kbps).
- The three lowest bits are used to identify different I2C masters on the same bus—each one has its unique identifier. During transmission of the

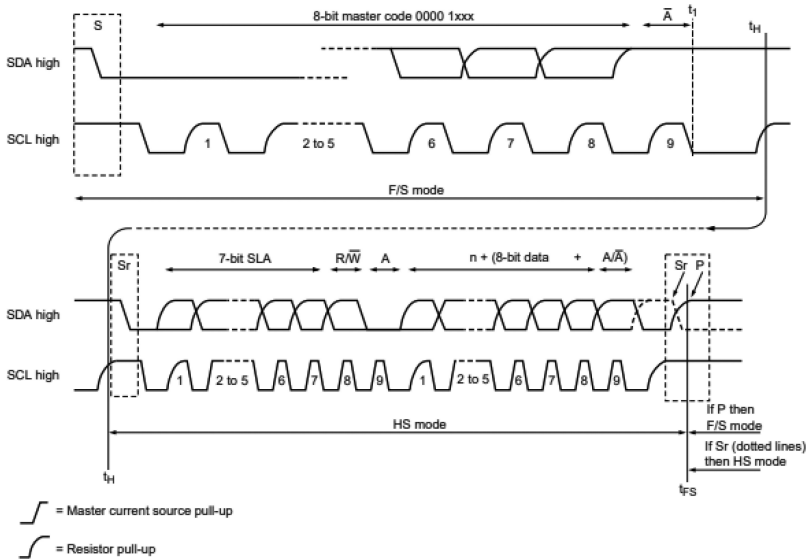


Figure 1.35 A complete HS-mode transfer.

master code, arbitration takes place (see arbitration latter), so that only the winning master can perform the following high-speed transfer. The winning master is the “active master.”

- The master codes are selectable by the designer and allow up to eight high-speed masters to be connected in one system (master code “00001000” should be reserved for test and diagnostic purposes).
- Active master switches to high-speed communication.
- Arbitration and clock synchronization only take place during master code transmission, not during HS transfer.
- After the acknowledge phase following the master code, the high-speed transfer begins with a repeated start condition, followed by the slave address and the succeeding data, just like in Fast or Standard mode, but with a higher bit rate.
- After each address, the slave must respond by *ack* or *nack* signal.
- The current source circuit is disabled after each repeated start condition and after each *ack* or *nack* to give slaves a chance to stretch the clock.
- The high-speed mode remains active until a stop condition is transmitted, on which the connected high-speed devices switch back to slow transmission rates like Fast or Standard mode.
- All devices return to fast mode operation after a stop condition is sent.

Note: Clock Stretching

Clock stretching during this mode has a special rule: It is only allowed after the ACK bit (and before the 1st bit of the next byte). Stretching between 2 and 9 bits is illegal because the edges of these bits are boosted with an additional current source.

1.18.5 Ultra-fast Mode (UFm)

The I2C Ultra-fast mode (*UFm*) is the fastest I2C mode available. It offers 5 MHz. This mode is pretty different to other I2C modes:

- UFm works **unidirectionally**, and data are sent in one direction only.
- Ultra-fast mode lines are driven in push–pull technology.
- Slaves do not send an acknowledge bit.
- There is a single master only.
- No arbitration for multi-master environment is available.

Nevertheless, multiple devices can be addressed sharing an I2C UFm bus; 112 nodes are available.

Currently, there are just a couple of devices supporting this mode. NXP's PCU 9669 is a 3-channel bus controller with two 5 MHz UFm and one fast-mode plus (FM+) interface. Variations are available.

Since the ultra-fast mode bus basically focuses on LED devices, NXP comes with an LED controller PCU9655.

To have more information about UFm mode, NXP provides the application note “AN11250 *Practical guide for the use of ultra-fast mode I2C bus.*”

1.19 I2C as a Multi-master Bus: Bus Arbitration

So far we have seen the operation of the I2C bus from the master's point of view and using only one master on the bus. The I2C bus was originally developed as a multi-master bus. This means that more than one device initiating transfers can be active in the system.

When using only one master on the bus, there is no real risk of corrupted data, except if a slave device is malfunctioning or if there is a fault condition involving the SDA/SCL bus lines. This situation changes when using more than one master, for example, use of the 2 microcontrollers units CPU1 and CPU2 in Figure 1.36.

When CPU 1 issues a start condition and sends an address, all slaves will listen (including CPU 2 which at that time is considered a slave as well). If

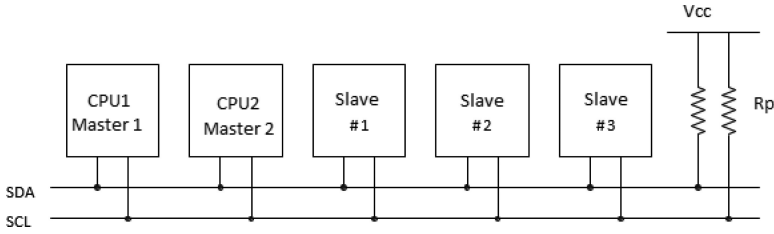


Figure 1.36 I2C bus interface—multiple master.

the address does not match the address of CPU 2, this device has to hold back any activity until the bus becomes idle again after a stop condition.

As long as the two CPUs monitor what is going on the bus (start and stop) and as long as they are aware that a transaction is going on because the last issued command was not a STOP, there is no problem. The problems arise if the two CPUs decide to initiate the “start condition” at the same time or if one of the CPUs miss the start condition and still think that the bus is idle. As a fact this is not the only problem that may arise when a system has multiple masters. A second problem that can arise is the case when having multiple clocks in the system. The first problem is resolved by “*arbitration*” and the second by “*synchronization*.” The two problems and their solution are discussed next.

1.19.1 Arbitration

For proper functioning in case of multi-master, each device needs to be able to cooperate with the fact that another device is currently talking and the bus is therefore busy. This can be translated into (Figure 1.37):

- **Being able to follow arbitration logic.** If two devices start to communicate at the same time, the one writing more zeros to the bus (or the slower device) wins the arbitration and the other device immediately discontinues any operation on the bus.
- **Bus busy detection.** Each device must detect an ongoing bus communication and must not interrupt it. This is achieved by recognizing traffic and waiting for a stop condition to appear before starting to talk on the bus.

The physical I2C bus setup is designed not only to help the devices to monitor what is going on the bus but more importantly, it is designed to prevent any risk of data corruption that may arise from data collision. The bus monitoring and collision avoidance are discussed next.

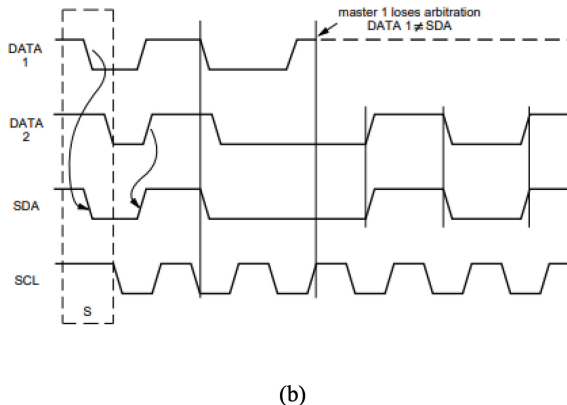
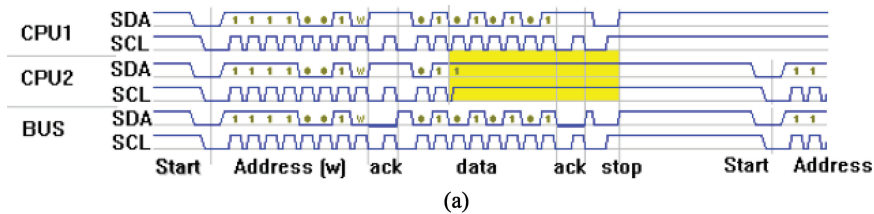


Figure 1.37 Bus arbitration.

There is an undefined condition if the arbitration procedure is still in progress at the moment when one master sends a repeated START or a STOP condition while the other master is still sending data. In other words, the following combinations result in an undefined condition:

- Master 1 sends a repeated START condition and master 2 sends a data bit.
- Master 1 sends a STOP condition and master 2 sends a data bit.
- Master 1 sends a repeated START condition and master 2 sends a STOP condition.

1.19.2 Bus Monitoring

The I2C bus structure is a wired AND. This means that if one device pulls a line low, it stays low and accordingly any device can test if the bus is idle or occupied. When a master (the sender) changes the state of a line to HIGH, it MUST always check that the line really has gone to HIGH. If it stays low, then this is an indication that the bus is occupied and some other device is pulling the line low.

Therefore, the general rule of thumb in I2C bus is: If a master (a sender) cannot get a certain line to go high, it loses arbitration and needs to back off and wait until a stop condition is seen before making another attempt to start transmitting.

1.19.3 Possibility of Collision

Since the previous rule says that a master loses arbitration when it cannot get either SCL or SDA to go high when needed, the problem of data corruption (or data collision) does not exist. It is the device that is sending the “0” that rules the bus. One master cannot disturb the transmission of other master because if it cannot detect one of the lines to go high, it backs off, and if it is the other master that cannot do so, it will behave the same.

This kind of back-off condition will only occur if the two levels transmitted by the two masters are not the same. As an example, let us consider Figure 1.30, where two CPUs start transmitting at the same time.

The two CPUs are accessing a slave in write mode at address 1111001. The slave acknowledges this. So far, both masters are under the impression that they “own” the bus. Now CPU1 wants to transmit 01010101 to the slave, while CPU2 wants to transmit 01100110 to the slave. The moment the data bits do not match anymore (because what the CPU sends is different than what is present on the bus), one of them loses arbitration and backs off. Obviously, this is the CPU which did not get its data on the bus. For as long as there has been no STOP present on the bus, it won’t touch the bus and leave the SDA and SCL lines alone (shaded zone). The moment a STOP was detected, CPU2 can attempt to transmit again.

From the example above, we can conclude that it is the master that is pulling the line LOW in an arbitration situation that always wins the arbitration. The master which wanted the line to be HIGH when it is being pulled low by the other master loses the bus. We call this a loss of arbitration or a back-off condition. When a CPU loses arbitration, it has to wait for a STOP condition to appear on the bus. Then, it knows that the previous transmission has been completed.

1.19.4 Clock Synchronization and Handshaking

1.19.4.1 Clock synchronization

All masters generate their own clock on the SCL line to transfer messages on the I2C bus. Data are only valid during the HIGH period of the clock. A

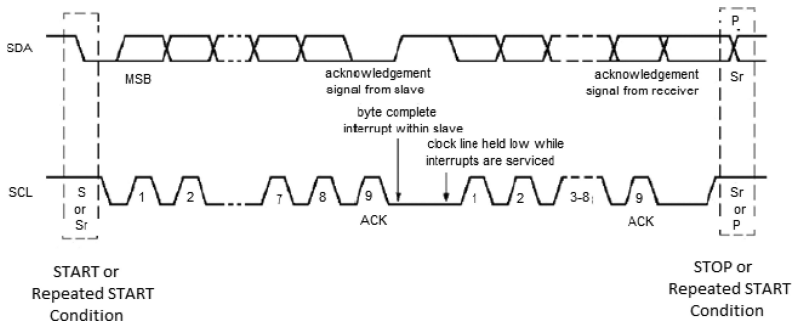


Figure 1.38 Clock synchronization.

defined clock is therefore needed for the bit-by-bit arbitration procedure to take place.

Clock synchronization is performed using the wired-AND connection of I2C interfaces to the SCL line. This means that a HIGH-to-LOW transition on the SCL line will cause the devices concerned to start counting off their LOW period, and once a device clock has gone LOW, it will hold the SCL line in that state until the clock HIGH state is reached. However, the LOW-to-HIGH transition of this clock may not change the state of the SCL line if another clock is still within its LOW period. The SCL line will therefore be held LOW by the device with the longest LOW period. Devices with shorter LOW periods enter a HIGH wait-state during this time, see Figure 1.38.

When all devices concerned have counted off their LOW period, the clock line will be released and go HIGH. There will then be no difference between the device clocks and the state of the SCL line, and all the devices will start counting their HIGH periods. The first device to complete its HIGH period will again pull the SCL line LOW. In this way, a synchronized SCL clock is generated with its LOW period determined by the device with the longest clock LOW period, and its HIGH period determined by the one with the shortest clock HIGH period.

1.19.4.2 Handshaking: Using the clock synchronizing mechanism as a handshake

The I2C protocol also includes a synchronization mechanism, which can be used as a handshake mechanism between slow and fast devices or between masters in a multi-master session.

When a slow slave (slow in terms of internal execution) is attached to the bus, then problems may occur. Let us consider a serial EEPROM. The actual writing process inside the EEPROM might take some time. Now if you send

multiple bytes to such a device, the risk exists that you send new data to it before it has completed the write cycle. This would corrupt the data or cause data loss.

The slave must have some means to tell the master that it is busy. It could of course simply not respond to the acknowledge cycle. This would cause the master to send a stop condition and retry. (That is how it is done in hardware in EEPROMs.) Other cases might not be so simple. Think about an A/D converter.

It might take some time for the conversion to complete. If the master would just go on, it would be reading the result of the previous conversion instead of the newly acquired data.

Now the synchronization mechanism can come in handy. This mechanism works on the SCL line only. The slave that wants the master to wait simply pulls the SCL low as long as needed. This is like adding “wait states” to the I2C bus cycle. The master is then not able to produce the acknowledge clock pulse because it cannot get the SCL line to go high. Of course, the master software must check this condition and act appropriately. In this case, the master simply waits until it can get the SCL line to go HIGH and then just goes on with whatever it was doing.

There are a number of minor drawbacks involved when implementing this.

If the SCL gets stuck due to an electrical failure of a circuit, the master can go into deadlock. Of course, this can be handled by timeout counters. Plus, if the bus gets stuck like this, the communication is not working anyway.

Another drawback is speed. The bus is locked at that moment. If you have rather long delays (long conversion time in our example above), then this penalizes the total bus speed a lot. Other masters cannot use the bus at that time either.

This technique does not interfere with the previously introduced arbitration mechanism because the low SCL line will lead to back-off situations in other devices which possibly would want to “claim” the bus. So, there is no real drawback to this technique except the loss of speed / bandwidth and some software overhead in the masters.

It is possible to use this mechanism between masters in a multi-master environment. This can prevent other master from taking over the bus. In a two-master system, this is not useful. But as soon as the system has three or more masters, this is very handy. A third master cannot interrupt a transfer between master 1 and 2 in this way. For some mission-critical situations, this can be a very nice feature.

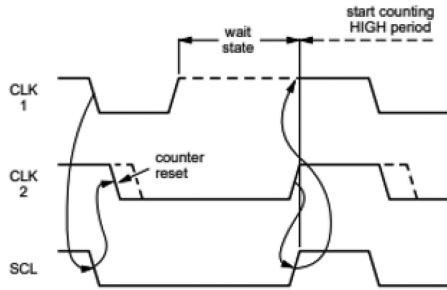


Figure 1.39 Clock synchronization during the arbitration procedure.

It is possible to make this technique more rigid by pulling not only the SCL line low but also the SDA line. Then, any master other than the two masters talking to each other will immediately back off. Before continuing, the master must first let SDA go back high, and then SCL, representing a stop condition.

Any master which attempted to communicate in the meantime would have detected a back-off situation and would be waiting for a STOP to appear

1.20 I2C Interface: Connecting I2C Bus to a PC

The term “I2C interface” usually refers to means for connecting an I2C bus to a PC. However, there are certainly other I2C interfaces which provide connectivity for this universally used bus to non-PC devices.

There are a number of ways to attach the I2C lines to a PC. There is even a zero-cost solution available which takes advantage of the PC parallel port pins. A bit-banging driver is then used to toggle the signals to implement a simple I2C single master device. Provided that the parallel port of the used PC is tolerant enough with respect to various electrical characteristics, such approach may well work for hobbyists. Since it is quite unreliable, a direct (hardwareless) parallel port interface should never be applied in commercial applications.

Philips provided schematics for an adapter called *Single master*, which essentially works like a hardwareless solution but compensates for some parallel port-related issues by using a driver chip between the PC port pins and the I2C SDA and SCL lines. Such interface can only be operated as single master, and the maximum speed highly depends on the drivers and operating system of the PC. Since each bit is toggled individually, no timing

can be guaranteed under Windows or Linux with such connection. Another disadvantage is the potential danger of damaging the PC with electronic discharge or faulty connections of the external lines. This is particularly annoying if the parallel port functionality is integrated into a multifunctional non-replaceable chip as is the case with all modern PCs and notebooks.

A much more reliable, powerful and safe alternative are interfaces which provide their own I2C logic such as Connii MM 2.0 or Tracii XL 2.0. These devices are connected via USB; in addition to being able to operate in multi-master mode, they provide a number of important and helpful additional features. Since they communicate with the PC software on a more efficient level than bit banging, the transfer speed is considerably higher and the load on the PC is much lower.

It is important to discuss the considerations that the user has to look for when choosing a professional I2C interface. Some of the considerations are as follows:

- An I2C interface should not only consist of wiring to the parallel port of a PC. This approach can cause damage to your PC or notebook and is not very reliable.
- If the user is intending to use the interface with another I2C master simultaneously, he/she should opt for a multi-master interface in order to avoid arbitration conflicts on the bus.
- The interface that the user is going to choose should be capable of running at the same speed as the other I2C devices in his/her system.
- If the application has slaves which are implemented using a microcontroller or if the user know that a slave device requires clock stretching, he/she need to make sure that the selected interface supports it.
- Choose an interface which has a user interface and programming API which you can easily use and which is supported.
- If possible, an USB interface is preferable over a plug-in card for the user's PC. This reduces the cable length you need extra on the I2C bus.
- Choose a vendor who offers reliable support and long-term market commitment. As operating systems tend to change from time to time, you will need upgrades in order to be able to use such new versions.

Summary

- IIC/I2C is an 8-bit bidirectional synchronous serial communication protocol requiring only two wires for operation.

- The I2C bus consists of two open-drain lines: SDA (data) and SCL (clock).
- Several devices, being either *Master* or *Slave*, can be connected to the bus. The *Master* device must initiate the transfer and drive the clock line (SCL).
- I2C supports the standard speed of 100 kbps, up to a maximum speed of 3.4 Mbps.
- *Master* must generate unique Start and Stop conditions in order to mark the beginning and end of a transaction.
- The receiver must send the ACK bit after every byte that it receives, failing which the *Master* may either Stop the transaction or attempt a repeated Start.
- Every device connected to the I2C bus has either 7-bit or 10-bit address. An additional R/W' bit is added to the address by the *Master* to determine whether it wants to read or write from/to the device.
- Data transfer can be unidirectional (*Master* to *Slave* OR vice versa) or bidirectional.
- *Slave* can hold the clock line low until it is ready with the result to be sent to the *Master*, called Clock Stretching.

Improved Inter-integrated Circuits (I3C)

MIPI Alliance Specification for I3CSM (Improved Inter Integrated Circuit), version 1.0, MIPI Alliance, Inc., MIPI Board adopted 31 December 2016.

MIPI Alliance Specification for I3C Host Controller Interface (I3C HCISM), version 1.0, MIPI Alliance, Inc., In press.

MIPI Alliance Specification for Discovery and Configuration (DisCoSM), version 1.0, MIPI Alliance, Inc., MIPI Board adopted 28 December 2016.

MIPI Alliance DisCoSM Specification for I3CSM, version 1.0, MIPI Alliance, Inc., In press.

MIPI Alliance Application Note for I3CSM, version 1.0, MIPI Alliance, Inc., In press.

MIPI Alliance Specification for Camera Serial Interface 2 (CSI-2SM), version 2.1, MIPI Alliance, Inc., In press.

MIPI Alliance Specification for Debug for I3CSM, version 1.0, MIPI Alliance, Inc., In press.

MIPI Alliance Specification for Virtual GPIO Interface (VGISM), version 1.0, MIPI Alliance, Inc., In press.

MIPI Alliance Conformance Test Suite (CTS) for I3CSM v1.0, CTS version 1.0, MIPI Alliance, Inc., In press.

CAN Data Link Layer

Controller Area Network, Basics Protocols , Chips and Applications; Dr. Konrad Etschberger; ISBN 3 00-007376-0 (www.ixxat.com)

CAN Systems Engineering , From Theory to Practical Applications; Wolfhard Lawrenz, ISBN 0-387-94939-9

Nicolas Navet , Yeqiong Song , Franycoise Simonot-Lion , and Cedric Wilwert , "Trends in Automotive Communication Systems," Proceedings of the IEEE, Vol. 93, NO. 6, June 2005 Page(s): 1204–1223

Gabriel Leen and Donal Heffernan , "Expanding Automotive Electronic Systems," IEEE Computer Volume 35, Issue 1, Jan. 2002 Page(s):88–93

Jeffrey Cook , Ilya Kolmanovsky , David McNamara , Edward Nelson and K. Venkatesh Prasad , "Control, Computing and Comm

J.M. Eklund , J. Sprinkle , S. Sastry and T.R. Hansen , "Information Technology for Assisted Living at Home: building a wireless infrastructure for assisted living," IEEE-EMBS 2005 27th Annual International Conference of the Engineering in Medicine and Biology Society, Sept. 2005 page(s): 3931–3934.

CAN in Automation Website , <http://www.can-cia.org>, viewed September 2006.

Robert Bosch GmbH , "CAN Specification 2.0," 1991.

Holger Zeltwanger , "Time-Triggered Communication on CAN," SAE Paper 2002-01-0437, 2002.

Thomas Fuehrer , Robert Hugel , Florian Hartwich and Harald Weiler , "FlexRay–The Communication System for Future Control Systems in Vehicles," SAE Paper 2003-01-0110, 2003.

"CAN History" (<http://www.can-cia.de/index.php?id=161>). CAN in Automation.

Building Adapter for Vehicle On-board Diagnostic (<http://www.obddiag.net/adapter.html>), <http://obddiag.net>, accessed 2009-09-09

Comparison of Event-Triggered and Time-Triggered Concepts with Regard to Distributed Control Systems A. Albert, Robert Bosch GmbH Embedded World, 2004, Nrnberg <http://www.semiconductors.bosch.de/en/20/can/2-license.asp>

"CAN BUS MESSAGE FRAMES–Overload Frame, Interframe Space" (<http://rs232-rs485.blogspot.com/2009/11/can-bus-message-frames-overload.html>).

Controller Area Network, Basics Protocols, Chips and Applications; Dr. Konrad Etschberger; ISBN 3–00–007376–0 (www.ixxat.com)

CAN Systems Engineering, From Theory to Practical Applications; Wolfhard Lawrenz, ISBN 0–387–94939–9

MCP2551 Data Sheet , "High Speed CAN Transceiver", DS21667, Microchip Technology, Inc. AN754, "Understanding Microchip's CAN Module Bit Timing", DS00754, Microchip Technology,

Inc.

ISO-11898-2, "Road Vehicles—Interchange of Digital Information—Part 2: High Speed Medium Access Unit and Medium Dependant Interface", International Organization for Standardization.
CAN System Engineering, "From Theory to Practical Applications", Wolfhard Lawrenz, Springer.

ZigBee PRO and ZigBee SECURITY

ZigBee Alliance, ZigBee Specification Document 053474r17, 2008

Daintree Network, "Comparing ZigBee Specification Versions,"

www.daintree.net/resources/spec-matrix.php

"How Does ZigBee Compare with Other Wireless Standards?" www.stg.com/wireless/ZigBee-comp.html

Craig, William C. "ZigBee: Wireless Control That Simply Works," ZigBee Alliance, 2003

Ember Corporation, EmberZNet Application Developer's Reference Manual, 2008

IEEE Std 802.15.4 2003

IEEE Std 802.15.4 2006

Harsh Dhaka, "ZigBee Technology: Wireless Control that Simply Works?", Communications Design Conference, 2 October 2003

Jorjeta G. Jetcheva, Yih-Chun Hu, Amit Kumar Saha, and David B. Johnson. "Design and Evaluation of a Metropolitan Area Multitier Wireless Ad Hoc Network Architecture in Wireless Sensor Networks." In Proceedings of the Fifth IEEE Workshop on Mobile Computing Systems & Applications, Monterey, CA, pp. 32–37, 2004

ZigBee Alliance. ZigBee Overview. September 2003

Aaron Alva, DMCA security research exemption for consumer devices, October 28, 2016.

<https://www.ftc.gov/news-events/blogs/techftc/2016/10/dmca-security-research-exemption-consumer-devices>.

Wireless Mesh Networking ZigBee vs. DigiMesh. <https://www.digi.com>.

About ZigBee Protocol. <https://sites.google.com/site/xbeetutorial/xbeeintroduction/zigbee>.

Inc. ZigBee Alliance, ZigBee Specification 053474r20 (September 7, 2012).

ZigBee Specification 053474r06 (December 4, 2004).

ZigBee 3.0 Task Force, ZigBee: Securing the Wireless IoT(2017).

Luke Tutty, Guide to ZigBee 3.0, March 21, 2017.

<https://mmbnnetworks.atlassian.net/wiki/display/ITZ/Guide+to+Zigbee+3.0>.

Ricky A. Melgares, ZigBee Analysis and Security (May 31, 2011).

Bjorn Stelte and Gabi Dreo Rodosek, Thwarting Attacks on ZigBee—Removal of the KillerBee Stinger.

Jeff John Roberts, Light Bulbs Flash "SOS" in Scary Internet of Things Attack, November 03, 2016. <http://fortune.com/2016/11/03/light-bulb-hacking/>.

Darren Pauli, IoT worm can hack Philips Hue light bulbs, spread across cities, November 10, 2016.

<https://www.theregister.co.uk/2016/11/10/iotwormcanhackphilipshuelightbulbsspreadacrosscities/>.

Internet of Things Map. <https://p16.praetorian.com/iotmap>.

Inc. ZigBee Alliance, ZigBee 3.0 Stack User Guide JN-UG-3113 (October 5, 2016).

ZigBee 3.0 Devices User Guide JN-UG-3114 (December 1, 2016).

Tobias Zillner, ZigBee Exploited: the good, the bad, and the ugly (August 16 2015).

Joshua Wright, KillerBee API Documentation, 2009.

Michael Ashbridge, Eddystone-EID, 2016.

<https://github.com/google/eddytone/blob/master/eddytone-eid/tools/eidtools.py>

Bluetooth

- Bisikian, C. An overview of the Bluetooth wireless technology. *IEEE Commun. Mag.* 2001, 39, 86–94.
- Bluetooth Core Specification v5.0, Bluetooth Special Interest Group (SIG), Dec. 2016.
- Bluetooth Core Specification v4.1, Bluetooth Special Interest Group (SIG), Dec. 2013.
- S. (n.d.), Bluetooth low energy (BLE) enabled devices market volume worldwide, from 2013 to 2020 (in million units), 2015. [Online]. Available: <https://www.statista.com/statistics/750569/worldwide-bluetooth-low-energy-device-market-volume/>
- Harris, A., III ; Khanna, V. ; Tuncay, G. ; Want, R. ; Kravets, R. Bluetooth low energy in dense IoT environments. *IEEE Commun. Mag.* 2016, 54, 30–36.
- Darroudi, S.M. ; Gomez, C. Bluetooth low energy mesh networks: A survey. *Sensors* 2017, 17, 1467.
- Hayajneh, T. ; Almashaqbeh, G. ; Ullah, S. ; Vasilakos, A.V. A survey of wireless technologies coexistence in WBAN: Analysis and open research issues. *Wirel. Netw.* 2014, 20, 2165–2199.
- Sairam, K. ; Gunasekaran, N. ; Reddy, S.R. Bluetooth in wireless communication. *IEEE Commun. Mag.* 2002, 40, 90–96.
- Jordan, R. ; Abdallah, C.T. Wireless communications and networking: An overview. *IEEE Antennas Propag. Mag.* 2002, 44, 185–193.
- Ferro, E. ; Potorti, F. Bluetooth and Wi-Fi wireless protocols: A survey and a comparison. *IEEE Wirel. Commun.* 2005, 12, 12–26.
- Bluetooth Versions Walkthrough and Bluetooth 4.0 Low Energy Development Resources. Available online: <https://www.cnx-software.com/2013/06/05/bluetooth-versions-walkthrough-and-bluetooth-4-0-low-e>.
- Carettoni, L. ; Merloni, C. ; Zanero, S. Studying bluetooth malware propagation: The BlueBag project. *IEEE Secur. Priv.* 2007, 5, 17–25.
- Bluetooth SIG , “Specification of the Bluetooth system v1.1, Core”, December 2000.
- Bluetooth SIG , “Specification of the Bluetooth system v1.1, Profiles”, December 2000.
- Salutation Consortium , “Salutation Architecture Specification v2.1, Part 1”, 1999.
- Sun Microsystems , “Jini Architecture Specification v1.2”, December 2001.
- Microsoft Corporation , “Universal Plug and Play Device Architecture v1.0”, June 2000.
- Jennifer Bray and Charles F Sturman , “Bluetooth, Connect without cables”, Prentice Hall, 2001.
- Oliver Kasten and Marc Langheinrich , “First Experiences with Bluetooth in the Smart-Its Distributed Sensor Network”, Swiss Federal Institute of Technology, October 2001.
- Eugene A Gryazin , “Service Discovery in Bluetooth”, Helsinki University of Technology, 2000
- Choonhwa Lee and Sumi Helal , “Protocols for Service Discovery in dynamic and mobile networks”, University of Florida, September 2000.
- Bluetooth SIG , “Mapping Salutation Architecture APIs to Bluetooth Service Discovery Layer”, White paper, July 1999.
- Sun Microsystems , “Jini Architectural overview”, White paper, January 1999.
- John Rakesh , “UPnP, Jini and Salutation—A look at some popular coordination frameworks for future networked devices”, California Software Laboratories, June 1999.
- Salutation Consortium , “Salutation Lite, Find-And-Bind Technologies for mobile devices”, White paper, June 1999.
- Microsoft Corporation , “Understanding Universal Plug and Play”, White paper, June 2000.
- Steven Czerwinski , Ben Zhau , Todd Hodes , Anthony Joseph and Randy Katz , “An Architecture for a Secure Service Discovery Service”, University of California, Berkeley, 1999.
- E. Guttman , C. Perkins , J. Veizades and M. Day , “Service Location Protocol, Version 2” IETF, June 1999.
- M. Wahl , T. Howes , S. Kille , “Lightweight Directory Access Protocol (v3)” IETF, December 1997.

Erik Guttman , "Service Location Protocol: Automatic Discovery of IP Network Services" IEEE Internet Computing, vol 3, no.4, pp 71–80, August 1999.

Network Simulator web page , available 8/2/2002. <http://www.isi.edu/nsnam/ns/>

BlueHoc web page , available 8/2/2002. <http://oss.software.ibm.com/bluehoc/>

Bahar, I. , Frohm, E. , Gaona, C. , Hachtel, G. , Macii, E. , Pardo, A. , Somenzi, F. : Algebraic decision diagrams and their applications. *Formal Methods in System Design* 10(2/3), 171–206 (1997).

Basagni, S. , Bruno, R. , Petrioli, C. : Device discovery in Bluetooth networks: A scatternet perspective. In: *Proc. Networking 2002, LNCS*, vol. 2345, pp. 1087–1092. Springer (2002).

BlueHoc—An open-source Bluetooth simulator: <http://www-124.ibm.com/developerworks/opensource/bluehoc>

Bluetooth specification, version 1.2, Bluetooth SIG, 2003, www.bluetooth.com

Bryant, R. : Binary decision diagrams and beyond: Enabling technologies for formal verification. In: *Proc. International Conference on Computer-Aided Design (ICCAD'95)*, pp. 236–243 (1995).

Clarke, E. , Fujita, M. , McGeer, P. , McMillan, K. , Yang, J. , Zhao, X. : Multi-terminal binary decision diagrams: An efficient data structure for matrix representation. *Formal Methods in System Design* 10(2/3), 149–169 (1997).

Daws, C. , Kwiatkowska, M. , Norman, G. : Automatic verification of the IEEE 1394 root contention protocol with KRONOS and PRISM. *International Journal on Software Tools for Technology Transfer (STTT)* 5(2–3), 221–236 (2004).

Duflot, M. , Fribourg, L. , Herault, T. , Lassaigne, R. , Magniette, F. , Messika, S. , Peyronnet, S. , Piconny, C. : Probabilistic model checking of the CSMA/CD protocol using PRISM and APMC. In: *Proc. 4th Workshop on Automated Verification of Critical Systems (AVoCS'04)*, *Electronic Notes in Theoretical Computer Science*, vol. 128(6), pp. 195–214. Elsevier Science (2004).

Kasten, O. , Langheinrich, M. : First experiences with Bluetooth in the Smart-Its distributed sensor network. In: *Proc. PACT'01* (2001).

Kwiatkowska, M. , Norman, G. : Verifying randomized Byzantine agreement. In: D. Peled , M. Vardi (eds.) *Proc. Formal Techniques for Networked and Distributed Systems (FORTE'02)*, LNCS, vol. 2529, pp. 194–209. Springer (2002).

Kwiatkowska, M. , Norman, G. , Parker, D. : Controller dependability analysis by probabilistic model checking. In: *Proc. 11th IFAC Symposium on Information Control Problems in Manufacturing (INCOM'04)* (2004).

Kwiatkowska, M. , Norman, G. , Parker, D. : PRISM 2.0: A tool for probabilistic model checking. In: *Proc. 1st International Conference on Quantitative Evaluation of Systems (QEST'04)*, pp. 322–323. IEEE Computer Society Press (2004).

Kwiatkowska, M. , Norman, G. , Parker, D. , Sproston, J. : Performance analysis of probabilistic timed automata using digital clocks. In: K. Larsen , P. Niebert (eds.) *Proc. Formal Modeling and Analysis of Timed Systems (FORMATS'03)*, LNCS , vol. 2791, pp. 105–120. Springer-Verlag (2003).

Padgett, J. Bahr, M. Batra, M. Holtmann, R. Smithbey, L. Chen , and K. Scarfone , *Guide to Bluetooth Security: Recommendations of the National Institute of Standards and Technology* (Special Publication 800-121 Revision 2). USA: CreateSpace Independent Publishing Platform, 2017.

Haataja, K. Hyppönen, S. Pasanen , and P. Toivanen , *Bluetooth Security Attacks: Comparative Analysis, Attacks, and Countermeasures*, ser. SpringerBriefs in Computer Science. Springer Berlin Heidelberg, 2013. [Online]. Available: <https://books.google.com/books?id=bFm6BAAQBAJ>

D.-Z. Sun , Y. Mu , and W. Susilo , "Man-in-the-middle attacks on secure simple pairing in bluetooth standard v5.0 and its countermeasure," *Personal Ubiquitous Comput.*, vol. 22, no. 1, pp. 55–67, Feb. 2018. [Online]. Available: <https://doi.org/10.1007/s00779-017-1081>

National Institute of Standards and Technology . *Guide to Bluetooth Security: Recommendations of the National Institute of Standards and Technology; Special Publication 800-121 Revision 1*; National Institute of Standards and Technology: Maryland, MD, USA, 2008.

NIST Special Publication 800-121 Revision 2: *Guide to Bluetooth Security*. Available online: <https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-121r2.pdf>.

Gerez, S. *Implementation of Digital Signal Processing: Some Background on GFSK Modulation*.

Available online: <http://wwwhome.ewi.utwente.nl/~{gerezsh/sendfile/sendfile.php/gfsk-intro.pdf?sendfile=gfsk-intro.pdf>.

Bluetooth 5 FAQ: Everything You Need to Know . Available online:

<https://www.macworld.com/article/3262664/hardware/bluetooth-5-faq-everything-you-need-to-know.html>.

Nateq Be-Nazir Ibn, M. ; Tarique, M. Bluetooth security threats and solutions: A survey. Int. J. Distrib. Parallel Syst. 2012, 3, 127.

Panse, T. ; Panse, P. A Survey on Security Threats and Vulnerability attacks on Bluetooth Communication. Int. J. Comput. Sci. Inf. Technol. 2013, 4, 741–746.

Hassan, S.S. ; Bibon, S.D. ; Hossain, M.S. ; Atiquzzaman, M. Security Threats in Bluetooth Technology. Comput. Secur. 2017, 74, 308–322.

Zou, Y. ; Wang, X. ; Hanzo, L. A survey on wireless security: Technical challenges, recent advances and future trends. arXiv 2015, arxiv:1505.07919.

Shrivastava, M. Analysis of security risks in Bluetooth. Int. J. Comput. Acad. Res. 2012, 1, 88–95.

National Institute of Standards and Technology . Guide to Bluetooth Security; NIST 800-121-Rev 1; NIST: Gaithersburg, MD, USA, 2016.

Mitchell, A. The Car Whisperer: Eavesdrop On and Take Part in Nearby Bluetooth Conversations. 2005. Available online: <https://www.theinternetpatrol.com/the-car-whisperer-eavesdrop-on-and-take-part-in-nearby-bluetooth-conversations/>.

Dunning, J.P. Bluetooth Threat Taxonomy. Available online: https://vtechworks.lib.vt.edu/bitstream/handle/10919/76883/etd-10242010-163002_Dunning_JP_T_2010.pdf?sequence=1&isAllowed=y

Bluetooth Security . Available online: https://cs.stanford.edu/people/eroberts/courses/soco/projects/2003-04/wireless-computing/sec_bluetooth.shtml

Sen, J. Security and Privacy Challenges in Cognitive Wireless Sensor Networks. In Cognitive Radio Technology Applications for Wireless and Mobile Ad hoc Networks; Meghanathan, N. , Reddy, Y.B. , Eds.; IGI-Global: Hershey, PA, USA, 2013.

Shaked, Y. ; Wool, A. Cracking Bluetooth PIN. Available online:

<http://www.eng.tau.ac.il/~{yash/shakedwool-mobisys05/>

Saravanan, K. ; Vijayanand, L. ; Negesh, R.K. A Novel Bluetooth Man-In-The-Middle Attack Based on SSP using OOB Association model. arXiv, 2012, arxiv:1203.4649.

Moreno, A. ; Okamoto, E. BlueSnarf Revisited: OBEX FTP Service Directory Traversal. Available online: https://link.springer.com/content/pdf/10.1007%2F978-3-642-23041-7_16.pdf.

Ahmed, M. ; Musleh, A. ; Marouf, A. ; Mahmoud, A. ; Abu-Amara, M. Bluetooth Security. Available online: https://www.slideshare.net/ram_ari/bluetooth-security.

Becker, A. Bluetooth Security & Hacks; Seminar ITS Ruhr-Universitat Bochum SS2007; Ruhr University of Bochum: Bochum, Germany, 2007; Available online: https://gysc.urjc.es/~anto/ubicuos2/bluetooth_security_and_hacks.pdf.

Trifinite: BlueBump . Available online: https://trifinite.org/trifinite_stuff_bluebump.html.

BlueDump . Available online: https://trifinite.org/trifinite_stuff_bluedump.html.

Trifinite: BluePrinting . Available online: https://trifinite.org/trifinite_stuff_blueprinting.html.

The Attack Vector “BlueBorne” Exposes Almost Every Connected Device. Available online: <https://www.armis.com/blueborne/>

Tsira, V. ; Nandi, G. Bluetooth technology: Security issues and its prevention. Int. J. Comput. Appl. Technol. 2014, 5, 1833–1837.

Nawir, M. ; Amir, A. ; Yaakob, N. ; Lynn, O. Internet of Things (IoT): Taxonomy of Security Attacks. In Proceedings of the 2016 3rd International Conference on Electronic Design (ICED), Phuket, Thailand, 11–12 August 2016.

Haataja, K. Security Threats and Countermeasures in Bluetooth-Enabled Systems. Kuopio University Publications H. Business and Information Technology 13. 2009. Page 75. Available online: http://epublications.uef.fi/pub/urn_isbn_978-951-27-0111-7/urn_isbn_978-951-27-0111-7.pdf.

Z-Wave Technology

Zensys , 90310020x, Z-Wave Device Class Specification

WHITEPAPER: Compared, INSTEON

C. W. Badenhop , S. R. Graham , B. W. Ramsey , B. E. Mullins , L. O. Mailloux , "The Z-Wave routing protocol and its security implications" <https://doi.org/10.1016/j.cose.2017.04.004>

M. B. Yassein , W. Mardini and A. Khalil , "Smart homes automation using Z-wave protocol," 2016 International Conference on Engineering & MIS (ICEMIS), Agadir, 2016, pp. 1–6.

doi:10.1109/ICEMIS.2016.7745306

Sigma Designs . (n.d.). Z-Wave development Kit. Retrieved June 2013 , from Sigma Designs public web site: http://www.sigmadesigns.com/uploads/documents/zwave_dev_kit_br.pdf

"Introduction to the Z-Wave Security Ecosystem", available from Sigma Design, Inc., on the company website: <http://z-wave.sigmadesigns.com/wp-content/uploads/2016/08/Z-Wave-Security-White-Paper.pdf>

M. B. Yassein , W. Mardini and A. Khalil , "Smart homes automation using Z-wave protocol," 2016 International Conference on Engineering & MIS (ICEMIS), Agadir, 2016, pp. 1–6.

doi:10.1109/ICEMIS.2016.7745306

Smart Home Protocols: Comparison

Aston, Kevin. "That 'Internet of Things' Thing", Available:

<http://www.rfidjournal.com/articles/view?4986>, accessed on June 2, 2017 .

Wright, J. (2011). Practical ZigBee Exploitation Framework. toorcon 2011.

Sigma Design press release , "Sigma Designs Announces Break-Through Z-Wave(R) UL Component Recognition", <http://www.sigmadesigns.com/news/sigma-designs-announces-break-through-z-waver-ul-component-recognition>, accessed on June 2, 2017 .

M. Knight , "Wireless security–How safe is Z-wave?," in Computing & Control Engineering Journal, vol. 17, no. 6, pp. 18–23, Dec.-Jan. 2006. doi: 10.1049/cce:20060601

M. Al-Zyoud , T. Atkison and J. Carver , "An Overview of Emerging Privacy Issues in the Internet of Things," 2016 International Conference on Computational Science and Computational Intelligence (CSCI), Las Vegas, NV, 2016, pp. 212–217. doi: 10.1109/CSCI.2016.0047

C. W. Badenhop , S. R. Graham , B. W. Ramsey , B. E. Mullins , L. O. Mailloux , "The Z-Wave routing protocol and its security implications" <https://doi.org/10.1016/j.cose.2017.04.004>

Threadgroup Inc . website: <http://Threadgroup.org/What-is-Thread/Connected-Home>

H. Sharma and S. Sharma , "A review of sensor networks: Technologies and applications," 2014 Recent Advances in Engineering and Computational Sciences (RAECS), Chandigarh, 2014, pp. 1–4. doi: 10.1109/RAECS.2014.6799579

M. B. Yassein , W. Mardini and A. Khalil , "Smart homes automation using Z-wave protocol," 2016 International Conference on Engineering & MIS (ICEMIS), Agadir, 2016, pp. 1–6. doi: 10.1109/ICEMIS.2016.7745306

Sigma Designs . (n.d.). Z-Wave development Kit. Retrieved June 2013 , from Sigma Designs public web site: http://www.sigmadesigns.com/uploads/documents/zwave_dev_kit_br.pdf

M. Zareei , A. Zarei , R. Budiarto and M. A. Omar , "A comparative study of short range wireless sensor network on high density networks," The 17th Asia Pacific Conference on Communications, Sabah, 2011, pp. 247–252. doi: 10.1109/APCC.2011.6152813

"Introduction to the Z-Wave Security Ecosystem", available from Sigma Design, Inc., on the company website: <http://z-wave.sigmadesigns.com/wp-content/uploads/2016/08/Z-Wave-Security-White-Paper.pdf>

OpenZwave . (n.d.). Retrieved June 2013 , from open-zwave Google code site:

<https://code.google.com/p/open-zwave/>

International Telecommunication Union (ITU) . (2012). G.9959: Short range narrow-band digital radio communication transceivers–PHY and MAC layer specifications.

Texas Instruments . (2009, December). CC1110 Mini Development Kit. Retrieved from <http://www.ti.com/tool/cc1110dk-mini-868>

I. Chatzigiannakis and A. Strikos , “A decentralized intrusion detection system for increasing security of wireless sensor networks,” 2007 IEEE Conference on Emerging Technologies and Factory Automation (EFTA 2007), Patras, 2007, pp. 1408–1411. doi: 10.1109/EFTA.2007.4416949

Texas Instruments . (2013, May). SmartRF Studio. Retrieved from <http://www.ti.com/tool/smartrfstudio>

Morais, A. , & Cavalli, A. (2011). Route Manipulation Attack in Wireless Mesh Networks. Advanced Information Networking and Applications (AINA).

G. Gaubatz , J. P. Kaps , E. Ozturk and B. Sunar , “State of the art in ultra-low power public key cryptography for wireless sensor networks,” Third IEEE International Conference on Pervasive Computing and Communications Workshops, Kauai Island, HI, 2005, pp. 146–150. doi: 10.1109/PERCOMW.2005.76

S. Zamfir , T. Balan , I. Iliescu and F. Sandu , “A security analysis on standard IoT protocols,” 2016 International Conference on Applied and Theoretical Electricity (ICATE), Craiova, 2016, pp. 1–6. doi: 10.1109/ICATE.2016.7754665

M. B. Yassein , W. Mardini and A. Khalil , “Smart homes automation using Z-wave protocol,” 2016 International Conference on Engineering & MIS (ICEMIS), Agadir, 2016, pp. 1–6. doi: 10.1109/ICEMIS.2016.7745306

“The Thread Group Expands Influence through Partnership with ZigBee Alliance, TCLA and Innovation Enabler Award”, (Accessed June 7, 2017) <http://mysmahome.com/news/5905/the-thread-group-expands-influence-through-partnership-with-zigbee-alliance-tcla-and-innovation-enabler-award-2/>

HackersOnBoard . “Black Hat 2013–Honey, I’m Home!!–Hacking Z-Wave Home Automation Systems.” Online video clip. YouTube. YouTube, 19 Nov 2013. Web 1 Nov. 2016 . URL: <https://www.youtube.com/watch?v=KYaEQhvdc8>

M. B. Tamboli and D. Dambawade , “Secure and efficient CoAP based authentication and access control for Internet of Things (IoT),” 2016 IEEE International Conference on Recent Trends in Electronics, Information & Communication Technology (RTEICT), Bangalore, 2016, pp. 1245–1250. doi: 10.1109/RTEICT.2016.7808031

J. Singh , T. Pasquier , J. Bacon , H. Ko and D. Eysers , “Twenty Security Considerations for Cloud-Supported Internet of Things,” in IEEE Internet of Things Journal, vol. 3, no. 3, pp. 269–284, June 2016. doi: 10.1109/JIOT.2015.2460333

L. Nastase , “Security in the Internet of Things: A Survey on Application Layer Protocols,” 2017 21st International Conference on Control Systems and Computer Science (CSCS), Bucharest, Romania, 2017, pp. 659–666. doi: 10.1109/CSCS.2017.101

M. B. Yassein , M. Q. Shatnawi and D. Al-Zoubi , “Application layer protocols for the Internet of Things: A survey,” 2016 International Conference on Engineering & MIS (ICEMIS), Agadir, 2016, pp. 1–4. doi: 10.1109/ICEMIS.2016.7745303